



TRABAJO FIN DE MÁSTER

Clasificación de Géneros Musicales Utilizando Espectrogramas y Redes Convolucionales

Realizado por
María García Cáceres

Para la obtención del título de
**Máster en Ingeniería del Software: Cloud, Datos y Gestión
TI**

Dirigido por
Jose María Luna Romera

Convocatoria de Junio, curso 2024/25

Agradecimientos

Quiero agradecer a todas las personas que, de una forma u otra, me han acompañado y apoyado durante el desarrollo de este trabajo y a lo largo del máster.

A Rafa, Aitor, Dani y Álvaro, por ayudarme prácticamente desde el primer día, hacerme sentir parte del grupo y convertir este camino en algo mucho más agradable.

A mi familia, por su apoyo incondicional incluso en los momentos en los que solo tenía quejas. A mi padre, especialmente, por animarme a dar el paso de cursar el máster.

A mi tutor, José María, por su cercanía y dedicación, siempre dispuesto a resolver mis dudas y a orientarme con paciencia.

A Rubén, María y Reyes por confiar en mí incluso cuando ni yo misma lo hacía.

Y a Fernandito ¡Bienvenido a la familia, enano!

Gracias a todos.

Resumen

Este trabajo aborda la clasificación automática de fragmentos musicales en distintos géneros utilizando redes neuronales convolucionales y espectrogramas generados a partir de diversos archivos de audio. El objetivo principal del proyecto ha sido desarrollar y evaluar un modelo capaz de realizar clasificación multilabel, permitiendo que un mismo fragmento musical pueda pertenecer a varios géneros simultáneamente, lo que refleja la complejidad de la música actual.

En cuanto al análisis del estado del arte, se observa una clara evolución desde métodos tradicionales basados en la extracción manual de características hasta el uso de arquitecturas profundas y segmentación en fragmentos cortos de audio. Esta última estrategia, aplicada con éxito en otros datasets, ha demostrado mejorar la capacidad de los modelos para capturar patrones representativos y generalizar mejor, especialmente en tareas de clasificación multilabel.

En el proyecto se ha empleado el dataset *MagnaTagATune*, conocido por su diversidad y riqueza en etiquetas obtenidas a través de encuestas a la población, lo que ha supuesto un reto adicional debido al desbalanceo de clases y a la subjetividad del etiquetado. Este problema se ha mitigado mediante técnicas de agrupación de géneros y un preprocesamiento específico, que incluye la fragmentación de los audios en segmentos de 3 segundos y el uso de ventanas deslizantes, aumentando así la cantidad y variedad de ejemplos para el entrenamiento del modelo.

La metodología combina la generación de espectrogramas como representación visual del audio y el uso de una arquitectura de red neuronal convolucional optimizada, que integra técnicas como la normalización por lotes y el *dropout* para mejorar la generalización y evitar el sobreajuste. El rendimiento del modelo se ha evaluado utilizando métricas específicas para clasificación multilabel y validación cruzada, obteniendo resultados superiores a los reportados en estudios previos con enfoques similares.

Palabras clave: CNN, Red Convolucional, espectrograma, espectrograma de Mel, clasificación, clasificación multilabel, MagnaTagATune

Abstract

This work deals with the automatic classification of musical fragments in different genres using convolutional neural networks and spectrograms generated from different audio files. The main objective of the project has been to develop and evaluate a model capable of multilabel classification, allowing the same musical fragment to belong to several genres simultaneously, which reflects the complexity of today’s music.

Regarding the analysis of the state of the art, there is a clear evolution from traditional methods based on manual feature extraction to the use of deep architectures and segmentation in short audio fragments. The latter strategy, successfully applied in other datasets, has been shown to improve the models’ ability to capture representative patterns and to generalise better, especially in multi-label classification tasks.

The project has employed the *MagnaTagATune* dataset, known for its diversity and richness in labels obtained through population surveys, which has posed an additional challenge due to class imbalance and labelling subjectivity. This problem has been mitigated by genre clustering techniques and specific preprocessing, including the fragmentation of the audios into 3-second segments and the use of sliding windows, thus increasing the number and variety of examples for model training.

The methodology combines the generation of spectrograms as a visual representation of the audio and the use of an optimised convolutional neural network architecture, which integrates techniques such as batch normalisation and dropout to improve generalisation and avoid overfitting. The performance of the model has been evaluated using specific metrics for multilabel classification and cross-validation, obtaining results superior to those reported in previous studies with similar approaches.

Keywords: CNN, Convolutional Network, spectrogram, Mel-spectrogram, classification, multilabel classification, MagnaTagATune

Índice general

1. Introducción	1
1.1. Introducción	1
1.2. Estructura de este documento	1
2. Fundamentos teóricos	3
2.1. Introducción	3
2.2. Inteligencia Artificial	3
2.3. Aprendizaje Automático	3
2.3.1. Técnicas de Aprendizaje Supervisado	3
2.3.2. Técnicas de Aprendizaje No Supervisado	4
2.4. Clasificación	5
2.5. Clasificación multilabel	6
2.6. Redes neuronales convolucionales	6
2.6.1. Capas convolucionales (Conv2D)	7
2.6.2. MaxPooling	7
2.6.3. Batch Normalization	8
2.6.4. Dropout	9
2.6.5. Capas densas (Dense)	9
2.6.6. Funciones de activación	9
2.7. Métricas de evaluación	10
2.7.1. Accuracy	10
2.7.2. Curva ROC	10
2.7.3. AUC (Área Bajo la Curva ROC)	11
2.7.4. Precisión (Precision)	12
2.7.5. Recall	12
2.7.6. F1 Score	13
2.8. Técnicas de validación y entrenamiento	13
2.8.1. Validación cruzada (K-Fold)	13
2.8.2. EarlyStopping	13
2.8.3. Sobreajuste (Overfitting)	14
2.9. Técnica de optimización	14
2.9.1. Método de aceleración de Nesterov	14
2.10. Representaciones espectrales del audio	14
2.10.1. Espectrograma	14
2.10.2. Escala de Mel	15
2.10.3. Espectrograma de Mel	15
3. Estado del Arte	17
3.1. Introducción	17
3.2. Enfoques Tradicionales	17
3.3. Segmentación en Fragmentos de 3 Segundos	18
3.4. Redes Neuronales y Representación de Audio	18

3.5. Redes Generativas y Técnicas de Deep Learning	19
3.6. Optimizadores y Arquitecturas	19
3.7. Antecedentes	20
3.8. Conclusiones	21
4. Estudio previo	23
4.1. Introducción	23
4.2. Objetivos	23
4.3. Metodología	23
4.3.1. Documentación	24
4.3.2. Dataset Utilizado en el Estudio	24
4.3.3. Gestión del almacenamiento de los espectrogramas	25
4.3.4. Implementación	26
4.3.5. Elección de algoritmos	26
4.3.6. Entrenamiento y evaluación	27
4.4. Planificación	27
4.4.1. Metodología de Trabajo	27
4.4.2. Planificación y fases del proyecto inicial	28
4.4.3. Planificación y fases del proyecto real	29
4.5. Presupuesto	31
5. Implementación	33
5.1. Introducción	33
5.2. Análisis exploratorio de los datos	33
5.3. Criterios de clasificación de géneros musicales	34
5.4. Visualización	35
5.5. Generación de espectrogramas	38
5.6. Entrenamiento	41
6. Validación	46
6.1. Introducción	46
6.2. Validación del modelo	46
6.3. Comparativa con resultados de estudios previos	49
7. Conclusiones y trabajo a futuro	50
A. Bibliografía	52

Índice de figuras

2.1. Técnicas de Aprendizaje Supervisado [41]	4
2.2. Técnicas de Aprendizaje No Supervisado [41]	5
2.3. Esquema de las CNN por Aphex [2]	6
2.4. Explicación curva ROC [40]	11
2.5. Explicación área bajo la curva ROC [40]	12
2.6. Ejemplo de espectrograma	15
2.7. Ejemplo de espectrograma de Mel	16
5.1. Distribución canciones por género	36
5.2. Correlación entre géneros	37
5.3. Número de géneros por canción	38
5.4. Ejemplos de espectrogramas generados	40
5.5. Redimensionamiento de los espectrogramas	41
5.6. Distribución de valores de píxeles en un espectrograma	42
5.7. Capas CNN	43
6.1. K-Fold Cross Validation	46
6.2. Evolución de las métricas de validación por época	48

Índice de extractos de código

5.1. Modelo CNN	44
5.2. Entrenamiento con early stopping	44

1. Introducción

1.1. Introducción

La música, como forma de expresión cultural y artística, ha evolucionado enormemente con el avance de la tecnología digital. Hoy en día, la enorme cantidad de contenido musical disponible en plataformas digitales plantea retos importantes para su organización, búsqueda y recomendación. En este contexto, la clasificación automática de música por géneros o etiquetas se ha convertido en una tarea fundamental para facilitar la gestión y el acceso a grandes colecciones musicales.

El procesamiento de señales de audio y la extracción de características relevantes son pasos clave para abordar esta tarea. Entre las diversas representaciones posibles, los espectrogramas destacan por ofrecer una visualización tiempo-frecuencia que captura la dinámica y textura del sonido. Esta representación permite aprovechar técnicas de visión como las redes neuronales convolucionales, que han demostrado gran eficacia en la detección de patrones complejos en imágenes y, por extensión, en espectrogramas musicales.

Este trabajo se centra en el diseño y evaluación de un modelo basado en CNN para la clasificación multilabel de fragmentos musicales, utilizando espectrogramas generados a partir de archivos de audio. La elección de un enfoque multilabel responde a la naturaleza multifacética de la música, donde una pieza puede pertenecer simultáneamente a varios géneros o estilos, lo que añade complejidad al problema y requiere una evaluación cuidadosa mediante métricas específicas.

Para el entrenamiento y validación del modelo se ha utilizado el dataset *Magna-TagATune*, reconocido por su diversidad y riqueza en etiquetas musicales. Este dataset presenta desafíos propios, como el desbalanceo en la distribución de clases. Además, se ha implementado una estrategia de validación cruzada y mecanismos para evitar el sobreajuste, garantizando la robustez de los resultados obtenidos.

1.2. Estructura de este documento

A continuación, se describen los contenidos de cada capítulo:

- **Introducción:** Se presenta el contexto general del trabajo, los objetivos principales y la motivación que impulsa el estudio. También se justifica la relevancia de aplicar técnicas de deep learning a la clasificación musical.
- **Fundamentos teóricos:** Se exponen los conceptos esenciales para comprender el contenido posterior. Se abordan temas como inteligencia artificial, aprendizaje automático (supervisado y no supervisado), clasificación (incluyendo clasificación multilabel) y redes neuronales convolucionales. Además, se detallan las métricas

de evaluación, técnicas de validación y entrenamiento, y las representaciones espectrales del audio utilizadas.

- **Estado del arte:** Se analiza la literatura relacionada, incluyendo enfoques tradicionales y modernos en la clasificación musical. Se destacan aspectos como la segmentación de audio, el uso de redes neuronales, arquitecturas relevantes, y se contextualiza el trabajo respecto a investigaciones previas.
- **Estudio previo:** Se describe un estudio preliminar realizado antes del desarrollo principal. Se incluyen los objetivos, la metodología utilizada, la descripción del dataset, la gestión de espectrogramas, el entorno tecnológico empleado y la planificación temporal del proyecto.
- **Implementación:** Se detalla el proceso de desarrollo del modelo propuesto. Incluye el análisis exploratorio de los datos, los criterios de clasificación utilizados, la generación de espectrogramas, el proceso de entrenamiento del modelo y otros aspectos técnicos relevantes.
- **Validación:** Se presentan los resultados obtenidos durante la validación del modelo. Se realiza una comparativa con estudios previos para evaluar el rendimiento y se discuten los hallazgos obtenidos.
- **Conclusiones y trabajo futuro:** Se reflexiona sobre los resultados alcanzados, las principales dificultades encontradas y las posibles líneas de mejora e investigación para trabajos futuros.

2. Fundamentos teóricos

2.1. Introducción

En esta sección se explican los conceptos teóricos que se han utilizado durante el desarrollo del modelo, tanto en la parte de entrenamiento como en la evaluación y validación. Para ello, comenzamos con una introducción a la Inteligencia Artificial, pasando por los fundamentos del aprendizaje automático y sus principales enfoques. Finalmente, se detallan conceptos específicos como la clasificación y la clasificación multilabel.

2.2. Inteligencia Artificial

La Inteligencia Artificial (también conocida por sus siglas IA) es un área de la informática que busca crear programas o sistemas capaces de realizar tareas que normalmente requieren inteligencia humana. Estas tareas incluyen entender el lenguaje, tomar decisiones, aprender de la experiencia, reconocer imágenes o sonidos [9].

La Unión Europea define un sistema de IA como un software que puede recibir información de su entorno, procesarla, y tomar decisiones o acciones de manera autónoma para cumplir ciertos objetivos [9]. Es decir, un sistema de IA no se limita a seguir instrucciones fijas, sino que puede adaptarse y actuar según la situación, lo cual lo hace útil en aplicaciones como asistentes virtuales, coches autónomos o sistemas de recomendación.

2.3. Aprendizaje Automático

El aprendizaje automático (o Machine Learning, ML) es una técnica dentro de la inteligencia artificial que permite que los sistemas aprendan por sí mismos a partir de los datos. En lugar de programar reglas específicas, se le dan ejemplos al sistema, y este aprende a identificar patrones y tomar decisiones por su cuenta [19].

Por ejemplo, si queremos que una máquina aprenda a distinguir correos electrónicos importantes de los que son spam, le damos muchos ejemplos de ambos tipos. Con el tiempo, el sistema aprende por sí mismo a hacer esta distinción sin que tengamos que escribir todas las reglas posibles.

2.3.1. Técnicas de Aprendizaje Supervisado

El aprendizaje supervisado es un tipo de aprendizaje automático donde se entrena a un modelo con datos que ya están etiquetados [44]. Esto significa que cada ejemplo que

se le da al modelo incluye tanto la entrada (por ejemplo, una imagen) como la respuesta correcta (por ejemplo, si en la imagen hay un perro o un gato). De esta forma, el sistema aprende a relacionar las entradas con las salidas correctas. El aprendizaje supervisado se suele emplear en:

- **Clasificación:** En este tipo de problemas, el objetivo es predecir una categoría o etiqueta discreta. Por ejemplo, determinar si un correo electrónico es spam o no, o identificar a qué género musical pertenece una canción.
- **Regresión:** Aquí, la variable a predecir es un valor numérico continuo. Un caso típico sería estimar el precio de una vivienda o la temperatura de una ciudad en un día determinado.
- **Series temporales:** Se trabaja con datos recogidos en intervalos regulares de tiempo para predecir valores futuros basándose en patrones y tendencias pasadas. Por ejemplo, pronosticar las ventas mensuales o la evolución de la temperatura a lo largo del año.

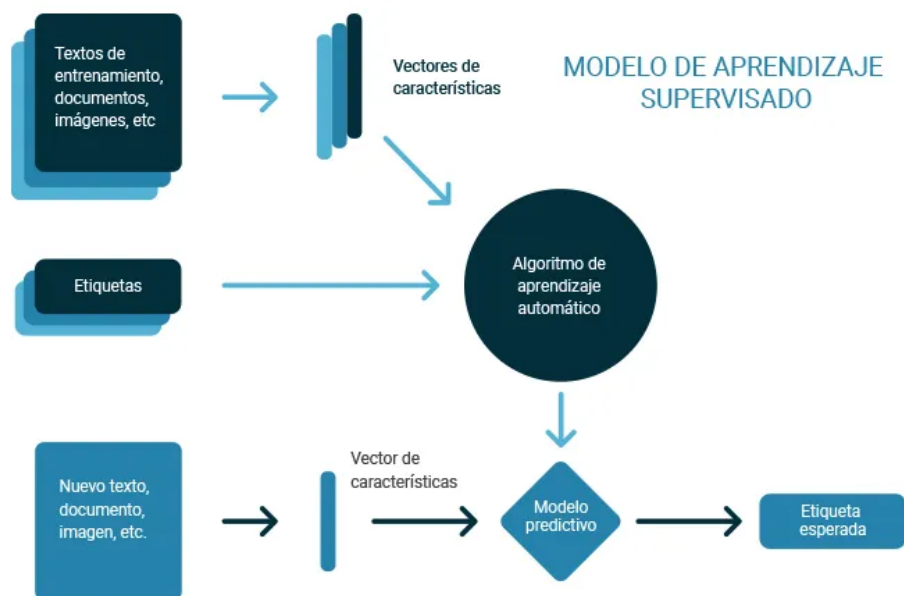


Figura 2.1: Técnicas de Aprendizaje Supervisado [41]

Por ejemplo, siguiendo el esquema de la figura superior 2.1, si queremos que un sistema reconozca letras escritas a mano (en el gráfico serían las etiquetas), le damos miles de imágenes de letras y le decimos cuál letra aparece en cada imagen. Con el tiempo, el sistema aprende a reconocer letras nuevas por sí solo.

2.3.2. Técnicas de Aprendizaje No Supervisado

IBM [24] define el aprendizaje no supervisado como aquel que se usa cuando no se tienen etiquetas. En este caso, el sistema intenta encontrar por sí mismo patrones ocultos en los datos. Por ejemplo, puede agrupar clientes con comportamientos similares

o detectar situaciones anómalas, como fallos en sensores, sin que nadie le diga lo que está “bien” o “mal”.

Este enfoque es muy útil cuando no es práctico o posible etiquetar grandes cantidades de información. En aplicaciones como el Internet de las Cosas (IoT), donde miles de dispositivos generan datos continuamente, los métodos no supervisados ayudan a detectar comportamientos anormales sin intervención humana.

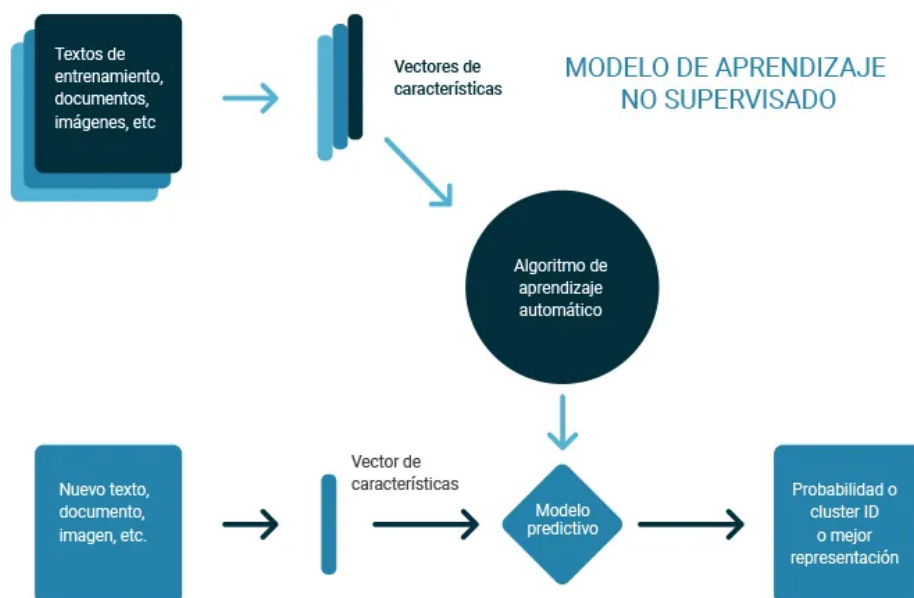


Figura 2.2: Técnicas de Aprendizaje No Supervisado [41]

Por ejemplo, siguiendo el esquema de la Figura superior 2.2, en una casa inteligente se pueden recopilar datos como el consumo eléctrico de los electrodomésticos (textos, registros, etc.), que se transforman en vectores de características. Estos vectores se procesan mediante un modelo de aprendizaje no supervisado, el cual identifica patrones normales de comportamiento sin necesidad de ejemplos etiquetados. Luego, un modelo predictivo utiliza estos patrones para analizar nuevos datos y asignarles una probabilidad o un identificador de grupo. De esta forma, si se detecta un consumo inusual, el sistema puede alertar sobre un posible problema sin haber visto antes ese caso específico.

2.4. Clasificación

Una de las aplicaciones más comunes del aprendizaje automático, tanto supervisado como no supervisado, es la clasificación. En ella, el modelo aprende a asignar una entrada a una o varias categorías. Por ejemplo, puede clasificar correos electrónicos como “spam” o “no spam”, o identificar emociones en una grabación de voz.

En este proyecto, la tarea principal consiste en clasificar muestras de audio según diferentes etiquetas, lo cual introduce una variación interesante de la clasificación tradicional: la clasificación multilabel.

2.5. Clasificación multilabel

En este proyecto, una misma muestra de audio puede tener varias etiquetas asociadas, es decir, puede pertenecer a varios géneros. Esto se llama clasificación *multilabel*, y es diferente de la clasificación tradicional, donde cada entrada solo pertenece a una clase.

En la clasificación multilabel, cada muestra puede estar asociada a múltiples etiquetas de forma simultánea, lo que implica que el modelo debe generar una salida que refleje la presencia o ausencia independiente de cada etiqueta. Esto se suele implementar aplicando una función de activación *sigmoide* (explicado en la sección 2.6.6) en la capa de salida. Además, la evaluación y el entrenamiento de estos modelos requieren métricas y funciones de pérdida que consideren la naturaleza multilabel, permitiendo optimizar cada etiqueta por separado y manejando correctamente la coexistencia de múltiples categorías en una misma instancia.

2.6. Redes neuronales convolucionales

Las redes neuronales convolucionales, conocidas como CNN por sus siglas en inglés, son un tipo de modelo de inteligencia artificial muy bueno para trabajar con imágenes. Funcionan especialmente bien cuando los datos tienen una estructura parecida a una rejilla, como ocurre con las imágenes digitales, que básicamente son una colección de píxeles organizados en filas y columnas [5]. Como se puede observar en la figura 2.3, las CNNs procesan la información a través de una secuencia de capas especializadas que transforman progresivamente los datos de entrada en representaciones más abstractas.

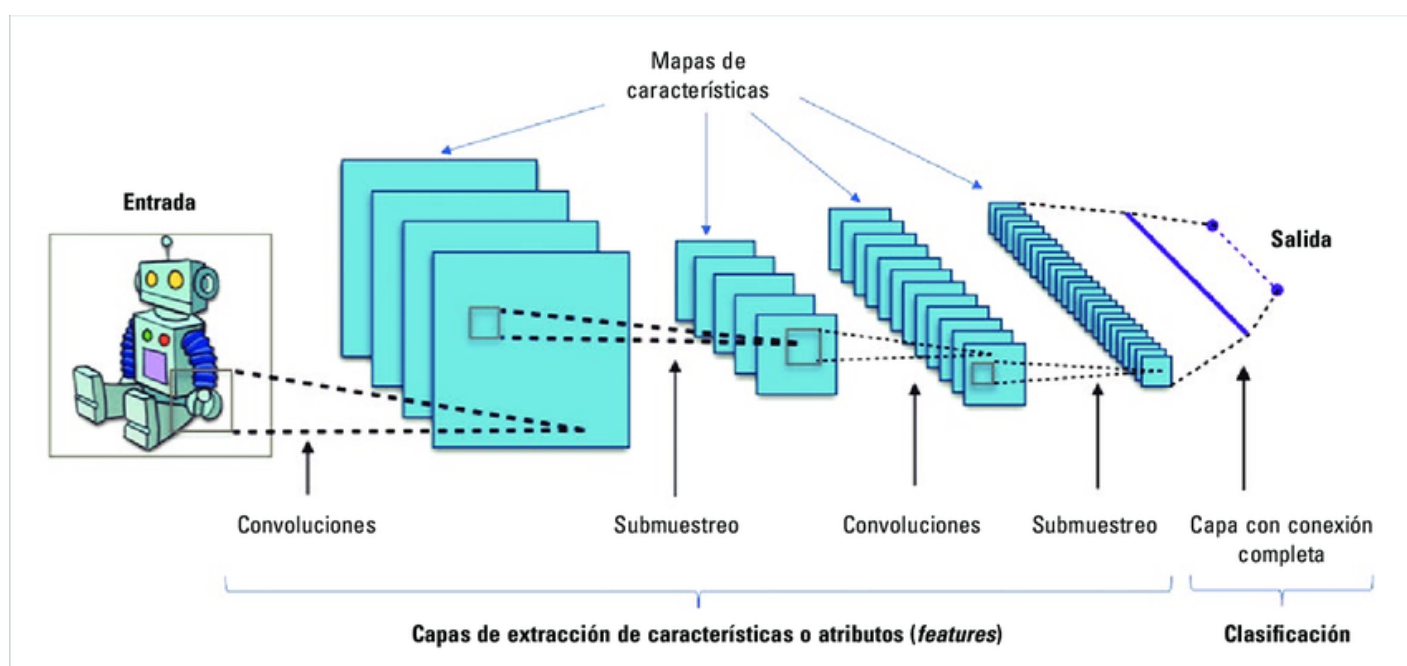


Figura 2.3: Esquema de las CNN por Aphex [2]

Estas redes están inspiradas en la forma en que el cerebro humano procesa lo que vemos, concretamente en cómo funciona la corteza visual. Gracias a esta inspiración biológica y su diseño técnico, las CNN se han convertido en una herramienta clave en muchas tareas de visión, como reconocer qué aparece en una imagen, localizar objetos dentro de ella o incluso dividirla por partes según su contenido [5].

Gracias a su estructura, las CNN están formadas por distintos tipos de capas, cada una con una función específica dentro del modelo. A continuación, se describen algunas de las más comunes y fundamentales para el funcionamiento de estas redes:

2.6.1. Capas convolucionales (Conv2D)

Las capas convolucionales son el núcleo de las redes neuronales convolucionales y su función principal es detectar características específicas dentro de una imagen, como bordes, texturas o formas[5]. Para lograrlo, utilizan filtros (también llamados kernels), que son pequeñas matrices de números que se van deslizando por la imagen, analizando fragmentos pequeños uno a uno.

Este proceso se conoce como convolución. Desde un punto de vista matemático, implica combinar los valores del filtro con los de la imagen en cada posición, y el resultado es una nueva imagen llamada mapa de características, donde se resaltan los patrones detectados.

En una dimensión, la operación de convolución entre una entrada x y un filtro w se representa así:

$$s(t) = (x * w)(t) = \left(\sum_{\tau} x(\tau) * w(t - \tau) \right)$$

En el contexto de imágenes bidimensionales, la convolución se extiende a dos dimensiones:

$$S(i, j) = (X * W)(i, j) = \sum_m \sum_n X(m, n) \cdot W(i - m, j - n)$$

Donde:

- X es la imagen de entrada.
- W es el filtro o kernel.
- S es la salida de la convolución.

Gracias a esta operación, la red puede aprender de forma automática qué patrones son importantes para la tarea que se le haya asignado, como identificar un objeto o clasificar una imagen [17].

2.6.2. MaxPooling

MaxPooling es una técnica que se utiliza para simplificar la información sin perder lo más importante. Su propósito es reducir el tamaño de las representaciones internas

(o mapas de características) generadas por las capas anteriores, lo que ayuda a que el modelo sea más eficiente y rápido [5].

El método funciona dividiendo la imagen (o el mapa de características) en pequeñas regiones que no se superponen, y dentro de cada una se toma únicamente el valor más alto. De esta manera, se conservan las características más destacadas de cada zona.

Matemáticamente, si tomamos una región R dentro de una entrada X , el resultado del MaxPooling se expresa así:

$$Y = \max_{(i,j) \in R} X(i,j)$$

Además de reducir el tamaño de los datos y la carga de cálculo, esta operación hace que la red sea más robusta frente a pequeños desplazamientos en la imagen, es decir, sigue reconociendo un objeto aunque esté levemente cambiado de lugar [5].

2.6.3. Batch Normalization

La normalización por lotes, conocida como Batch Normalization, es una técnica muy útil para mejorar el entrenamiento de redes neuronales profundas [5]. Su objetivo es hacer que el proceso de aprendizaje sea más rápido, más estable y más eficiente.

Lo que hace esta técnica es ajustar automáticamente los valores que salen de cada capa para que tengan una media cercana a cero y que su desviación estándar sea cercana a uno.

Para una activación x , la normalización se calcula así:

$$\hat{x} = \frac{x - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$$
$$y = \gamma \hat{x} + \beta$$

Donde:

- μ_B y σ_B^2 son la media y la varianza de los valores del mini-lote.
- ϵ es una constante pequeña que se añade para evitar errores de división por cero.
- γ y β son valores que la red aprende, y sirven para mantener la flexibilidad del modelo, permitiéndole ajustar la salida si lo necesita [5].

En resumen, esta técnica ayuda a que las redes aprendan mejor y más rápido, sin que los valores internos se descontrolen durante el entrenamiento.

2.6.4. Dropout

El Dropout es una técnica de regularización que previene el sobreajuste en una red neuronal. Durante el entrenamiento, se desactivan aleatoriamente un porcentaje de neuronas en cada capa, lo que obliga a la red a no depender excesivamente de ninguna neurona en particular [5]. Matemáticamente, para una neurona con activación h , el Dropout se aplica como:

$$\tilde{h} = h * r$$

Donde:

- r es una variable aleatoria de Bernoulli con probabilidad p de ser 1 (neurona activa) y $1 - p$ de ser 0 (neurona desactivada).

2.6.5. Capas densas (Dense)

Las capas densas, también conocidas como capas completamente conectadas, conectan cada neurona de la capa anterior con cada neurona de la capa actual. Son responsables de combinar las características extraídas por las capas anteriores para realizar tareas de clasificación o regresión [17]. La salida y de una capa densa se calcula como:

$$y = f(Wx + b)$$

Donde:

- x es el vector de entrada.
- W es la matriz de pesos.
- b es el vector de sesgos.
- f es la función de activación aplicada elemento a elemento.

2.6.6. Funciones de activación

Las funciones de activación introducen no linealidades en la red, permitiendo que aprenda representaciones complejas [17]. Las funciones principales son:

- **ReLU (Rectified Linear Unit):** Se define como $f(x) = \max(0, x)$. Es ampliamente utilizada por su simplicidad y eficacia en la mitigación del problema del desvanecimiento del gradiente [17].
- **Sigmoid:** Se define como $f(x) = \frac{1}{1+e^{-x}}$. Es útil en la capa de salida para tareas de clasificación multietiqueta, ya que produce valores entre 0 y 1, interpretables como probabilidades.

2.7. Métricas de evaluación

Las métricas de evaluación son fundamentales para medir el rendimiento de un modelo de inteligencia artificial, ya que permiten cuantificar cómo de bien se están cumpliendo los objetivos del sistema en tareas específicas.

En nuestro caso, vamos a centrarnos en aquellas métricas que permiten analizar la capacidad del modelo para clasificar correctamente los datos y manejar posibles desequilibrios entre clases. En particular, se utilizarán el accuracy, la curva ROC, el área bajo la curva (AUC), la precisión (precision), el recall y la métrica F1, ya que ofrecen una visión completa del comportamiento del modelo desde distintas perspectivas:

2.7.1. Accuracy

La accuracy mide la proporción de predicciones correctas realizadas por el modelo sobre el total de predicciones.

Fórmula:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Donde:

- TP: Verdaderos Positivos
- TN: Verdaderos Negativos
- FP: Falsos Positivos
- FN: Falsos Negativos

Esta métrica es útil en conjuntos de datos balanceados, pero hay que tener cuidado en casos en los que las clases estén desbalanceadas, ya que el modelo puede obtener un alto accuracy simplemente prediciendo la clase mayoritaria.

2.7.2. Curva ROC

La curva ROC es una representación gráfica que muestra la relación entre la tasa de verdaderos positivos (TPR) y la tasa de falsos positivos (FPR) a diferentes umbrales de clasificación.

Fórmulas:

- Tasa de Verdaderos Positivos (TPR):

$$TPR = \frac{TP}{TP + FN}$$

- Tasa de Falsos Positivos (FPR):

$$FPR = \frac{FP}{FP + TN}$$

La curva ROC permite visualizar el rendimiento del modelo en distintos puntos de decisión y es especialmente útil para comparar diferentes modelos.

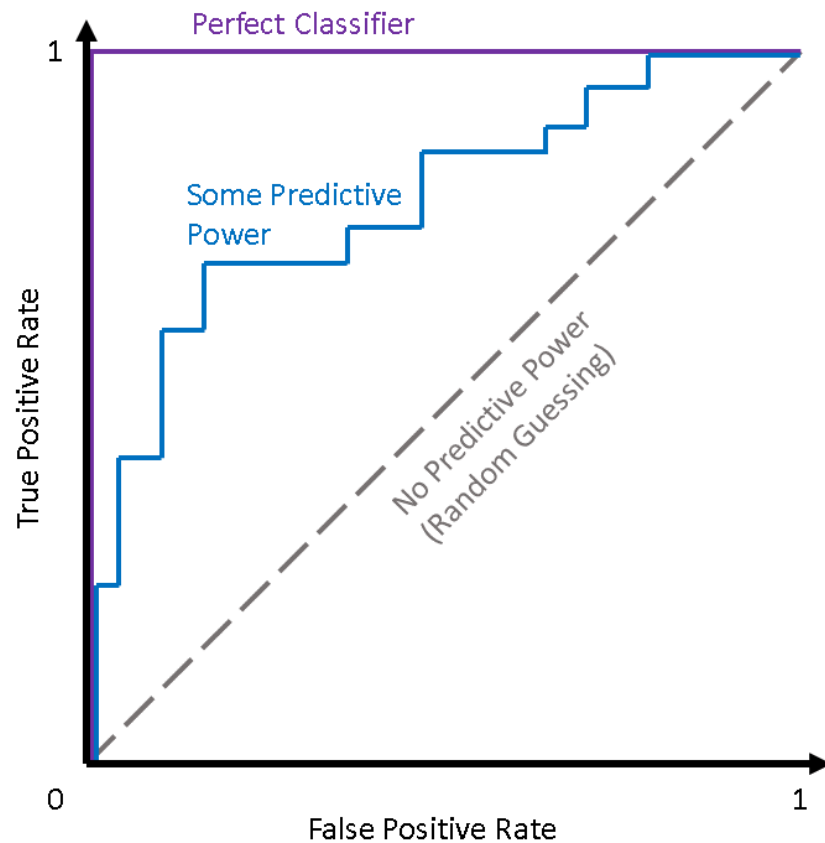


Figura 2.4: Explicación curva ROC [40]

Como se puede ver en la Figura 2.5, se utiliza como referencia la diagonal que representa un clasificador aleatorio, donde no existe capacidad predictiva real (es decir, se acierta por puro azar). Los modelos con cierto poder predictivo tienden a ubicarse por encima de esta diagonal, mientras que un clasificador ideal se acercaría al extremo superior izquierdo del gráfico, maximizando la tasa de verdaderos positivos y minimizando la de falsos positivos [40].

2.7.3. AUC (Área Bajo la Curva ROC)

El AUC o área bajo la curva ROC, cuantifica la capacidad del modelo para distinguir entre clases. Un AUC de 1.0 indica una clasificación perfecta, mientras que un AUC de 0.5 sugiere un rendimiento aleatorio [31]. Generalmente, cuanto mayor es el AUC, mejor es el desempeño del modelo en términos de separación entre las clases positivas y negativas.

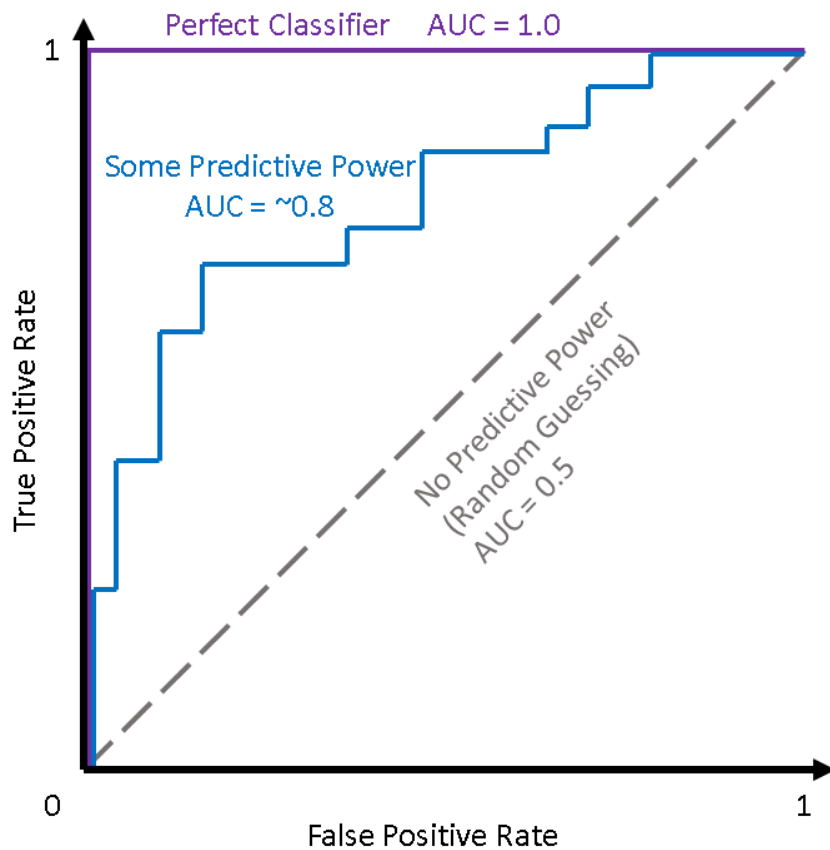


Figura 2.5: Explicación área bajo la curva ROC [40]

En la Figura 2.5 se puede observar cómo varía el valor del AUC según el poder predictivo del modelo: un clasificador aleatorio tiene un AUC de 0.5, mientras que un modelo con cierto poder predictivo puede alcanzar valores alrededor de 0.8. El clasificador perfecto alcanza el valor máximo de AUC, igual a 1.0. Esta métrica resulta especialmente útil porque resume en un solo número el rendimiento general del modelo, independientemente del umbral de decisión elegido [40].

2.7.4. Precisión (Precision)

Indica qué porcentaje de las etiquetas predichas fueron realmente correctas [17].

Fórmula:

$$Precision = \frac{TP}{TP + FP}$$

2.7.5. Recall

El recall mide capacidad del modelo para identificar todas las instancias positivas reales.

Fórmula:

$$Recall = \frac{TP}{TP + FN}$$

Es crucial en contextos donde es importante minimizar los falsos negativos.

2.7.6. F1 Score

El F1 Score es la media armónica de la precisión y el recall, proporcionando un equilibrio entre ambas métricas.

Fórmula:

$$F1Score = 2 * \frac{Precision * Recall}{Precision + Recall}$$

Es útil cuando se busca un balance entre precisión y recall, especialmente en conjuntos de datos desbalanceados.

2.8. Técnicas de validación y entrenamiento

Una vez tenemos definido el modelo y preparado el conjunto de datos, el siguiente paso es entrenarlo y validar su rendimiento. En esta sección se presentan estrategias clave como la validación cruzada, el uso de mecanismos de parada temprana y métodos para prevenir el sobreajuste e intentar conseguir un modelo robusto.

2.8.1. Validación cruzada (K-Fold)

La validación cruzada K-Fold consiste en dividir el conjunto de datos en k particiones del mismo tamaño. En cada iteración, una partición se utiliza como conjunto de validación y las restantes para entrenamiento [34]. Este proceso se repite k veces, asegurando que cada partición actúe como conjunto de validación una vez. Este método permite evaluar la capacidad de generalización del modelo y proporciona una estimación más robusta del rendimiento, especialmente en conjuntos de datos limitados.

2.8.2. EarlyStopping

El Early Stopping es una técnica de regularización que detiene el entrenamiento del modelo cuando el rendimiento en el conjunto de validación deja de mejorar durante un número predefinido de épocas [30]. Esta estrategia previene el sobreajuste (explicado en la sección 2.8.3) al evitar que el modelo aprenda patrones específicos del conjunto de entrenamiento que no generalizan bien a datos no vistos.

2.8.3. Sobreajuste (Overfitting)

El sobreajuste ocurre cuando un modelo aprende demasiado bien los detalles y el ruido del conjunto de entrenamiento, resultando en un rendimiento deficiente en datos nuevos [33]. Para mitigar este problema, se emplean técnicas de regularización como Dropout (explicado en la sección 2.6.4), que desactiva aleatoriamente neuronas.

2.9. Técnica de optimización

Una vez establecidas las estrategias de validación y entrenamiento, el siguiente paso es seleccionar una técnica de optimización que permita ajustar los parámetros del modelo de forma eficiente. La elección de un optimizador influye directamente en la velocidad y estabilidad del aprendizaje. A continuación, se presenta el método de aceleración de Nesterov, una técnica utilizada por su capacidad de mejorar la convergencia en redes neuronales profundas.

2.9.1. Método de aceleración de Nesterov

El método de aceleración de Nesterov, también conocido como Nesterov Accelerated Gradient (NAG), es una técnica de optimización que mejora el método del gradiente descendente mediante la incorporación del método del descenso del gradiente [37]. A diferencia del momentum clásico, donde la actualización se realiza basándose en la posición actual, NAG realiza un paso hacia adelante en la dirección del momentum antes de calcular el gradiente [13]. Esta anticipación permite al algoritmo corregir la dirección si se detecta que el gradiente cambia significativamente, resultando en una convergencia más rápida y estable.

2.10. Representaciones espectrales del audio

Una forma de representar señales de audio en tareas de procesamiento y análisis automático es mediante representaciones espectrales, que muestran cómo varía el contenido de frecuencia de una señal a lo largo del tiempo [14]. Estas representaciones permiten identificar características específicas como ritmos, timbres o patrones temporales, y son fundamentales en aplicaciones como clasificación musical, reconocimiento de voz y síntesis sonora [6].

2.10.1. Espectrograma

Como se explica en el trabajo de Paz et al. [38], el espectrograma se genera mediante la transformada de Fourier de tiempo corto (Short-Time Fourier Transform, STFT), que descompone la señal en ventanas temporales sucesivas para obtener su contenido frecuencial. El resultado es una representación bidimensional con el tiempo

en el eje horizontal, la frecuencia en el eje vertical y la intensidad codificada por el color. Dependiendo del objetivo, las frecuencias pueden representarse en escala lineal o logarítmica.

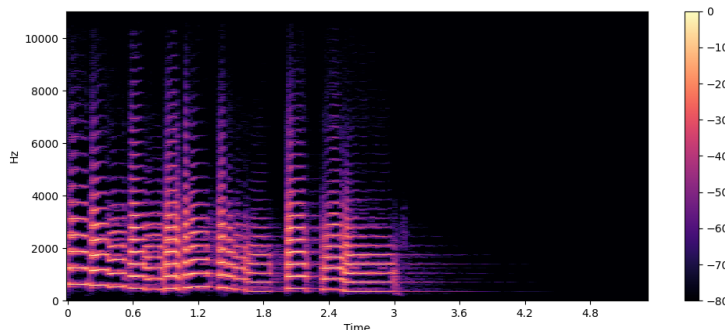


Figura 2.6: Ejemplo de espectrograma

Entre las variantes del espectrograma se encuentran el espectrograma de potencia, el logarítmico y el de Mel, cuya elección depende del tipo de tarea a realizar, como clasificación, segmentación o síntesis [39].

2.10.2. Escala de Mel

La escala de Mel es una escala perceptiva de frecuencias basada en cómo los seres humanos perciben los cambios de tono, en lugar de usar una escala física lineal. Estudios psicoacústicos muestran que el oído humano es más sensible a diferencias en frecuencias bajas que en altas, lo que motiva una transformación no lineal de las frecuencias [42]. La relación entre la frecuencia física y la frecuencia en Mels se expresa mediante la fórmula:

$$\text{Mel}(f) = 2595 \cdot \log_{10} \left(1 + \frac{f}{700} \right) \quad (2.1)$$

Esta transformación se ha convertido en un estándar en el procesamiento de audio, ya que ajusta mejor la representación espectral a la percepción humana [47].

2.10.3. Espectrograma de Mel

El espectrograma de Mel es una variante del espectrograma que utiliza un banco de filtros triangulares distribuidos según la escala de Mel, donde cada filtro simula una banda crítica del oído humano. El proceso consiste en aplicar la STFT a la señal, proyectar el resultado sobre la escala de Mel mediante los filtros, y finalmente aplicar una compresión logarítmica.

Esta representación es especialmente útil en tareas que involucran percepción auditiva, como reconocimiento de voz o clasificación de sonidos ambientales, y ha demostrado mejorar el rendimiento de modelos basados en redes neuronales convolucionales (CNN) [8, 21].

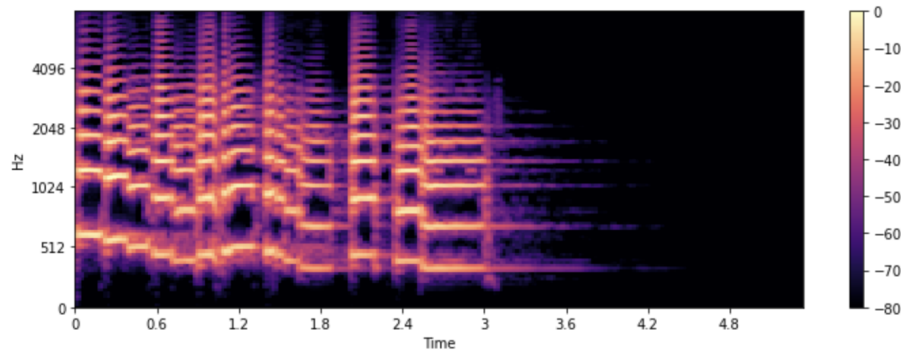


Figura 2.7: Ejemplo de espectrograma de Mel

3. Estado del Arte

3.1. Introducción

La música ha experimentado una transformación significativa en la era digital. El creciente volumen de datos musicales disponibles en formato digital y la diversidad de géneros y subgéneros han generado la necesidad de desarrollar herramientas automáticas que permitan la categorización de información de manera eficiente y precisa [12].

El campo de la clasificación automática de géneros musicales se ha beneficiado del avance en técnicas de procesamiento de señales, machine learning y deep learning [3]. Las primeras aproximaciones basadas en métodos estadísticos tradicionales han evolucionado hacia arquitecturas complejas de redes neuronales profundas, siendo capaces de capturar patrones contenidos dentro de los datos musicales. Estas técnicas han incrementado la precisión de los sistemas de clasificación, abriendo nuevas posibilidades para aplicaciones como los sistemas de recomendación o las plataformas de streaming musical.

Además, para mejorar el rendimiento de los sistemas de clasificación, la segmentación de audio y el preprocesamiento de datos han demostrado ser elementos cruciales [1]. En particular, el análisis de fragmentos cortos de audio ha permitido a los modelos identificar características relevantes de los géneros musicales, reduciendo la influencia de elementos irrelevantes o ruidosos presentes en grabaciones más largas [32].

En este estado del arte, se presentan las principales aproximaciones al problema de la clasificación automática de géneros musicales, abarcando desde métodos tradicionales hasta enfoques modernos basados en redes neuronales y técnicas generativas.

3.2. Enfoques Tradicionales

Los primeros sistemas automáticos de clasificación de géneros musicales se basaban en modelos de mezcla gaussiana (Gaussian Mixture Model, GMM) y modelos ocultos de Markov (Hidden Markov Model, HMM) para capturar características acústicas de los audios [36]. Sin embargo, estos métodos presentaban limitaciones, como una menor capacidad para modelar dependencias a largo plazo y relaciones no lineales entre los datos.

Bala Ganesh et al. [4] indica que el uso de coeficientes cepstrales de frecuencia Mel (Mel-Frequency Cepstral Coefficients, MFCC) destaca como técnica clave para la extracción de características. Los Mel-Frequency Cepstral Coefficients capturan las frecuencias más relevantes para la percepción humana, lo que los hace útiles en tareas de clasificación. Sin embargo, al depender de estadísticas globales, estas técnicas carecen de la capacidad de analizar patrones complejos que se extienden en el tiempo.

Otros trabajos, como el de Jain et al. [25], emplean representaciones en el dominio frecuencial para extraer características del audio y permitir la identificación de patrones específicos. Aunque estas técnicas han sido fundamentales, su aplicabilidad inicial se limitaba a escenarios con menor complejidad musical y conjuntos de datos reducidos.

3.3. Segmentación en Fragmentos de 3 Segundos

Uno de los descubrimientos más significativos en la clasificación automática de géneros musicales es la importancia de la duración de los fragmentos de audio utilizados en el análisis. En el trabajo de Ndou et al. [36], se demuestra que dividir grabaciones de 30 segundos en fragmentos más cortos, específicamente de 3 segundos, mejora notablemente el rendimiento de los modelos de clasificación. Además, la investigación de Dong [12] ya observó que la precisión de clasificación humana se estabiliza después de escuchar solo 3 segundos de música, sin mejorar significativamente con fragmentos más largos.

Dong [12] indica que el análisis de fragmentos más cortos se adoptó, en parte, como una estrategia de “divide y vencerás” porque no es factible alimentar el espectrograma completo de una señal musical de 30 segundos a una red neuronal convolucional (CNN) debido a su alta dimensión. Al dividir el audio original de 30 segundos en múltiples segmentos de 3 segundos, generalmente consecutivos y a menudo con solapamiento, se logra un conjunto de datos considerablemente más amplio y diversificado para el entrenamiento. Por ejemplo, el estudio de Ndou et al. [36] duplicó el conjunto de datos GTZAN de 1000 pistas de 30 segundos para crear 10,000 fragmentos de 3 segundos. Cada segmento puede tratarse como una instancia independiente para el entrenamiento del modelo, lo que aumenta la capacidad del modelo para generalizar y capturar patrones relevantes específicos del género.

En el conjunto de datos GTZAN [36], el uso de fragmentos de 3 segundos permitió que un clasificador basado en k-Nearest Neighbors (kNN) alcanzara una precisión del 92.69 %, superando significativamente los resultados obtenidos con grabaciones completas de 30 segundos y con otros clasificadores tradicionales. Otro estudio realizado por Bala Ganesh et al. [4] utilizando una arquitectura GenreNet con optimización Nadam logró una precisión del 84.50 % con fragmentos de 3 segundos de la base de datos GTZAN, en comparación con solo el 33.00 % utilizando las pistas completas de 30 segundos. Esta mejora generalizada se atribuye a que los fragmentos más cortos, y las características derivadas de ellos (como los mel-espectrogramas generados a partir de estos segmentos), capturan mejor las transiciones y variaciones locales en la música, esenciales para distinguir géneros musicales.

3.4. Redes Neuronales y Representación de Audio

Con la introducción de redes neuronales profundas, especialmente redes convolucionales (Convolutional Neural Networks, CNN), la clasificación de géneros musicales experimentó mejoras significativas en precisión y capacidad de generalizar [36, 4]. Las

Convolutional Neural Networks pueden procesar espectrogramas como si fueran imágenes, extrayendo patrones complejos de tiempo y frecuencia.

Para que las redes neuronales puedan procesar datos de audio, estos deben ser transformados en un formato adecuado. Una representación muy común y efectiva es el espectrograma, una imagen bidimensional que visualiza la intensidad de las frecuencias de una señal de audio a lo largo del tiempo [14]. Dentro de estas representaciones visuales, el mel-espectrograma destaca por imitar la percepción auditiva humana, dando mayor importancia a las frecuencias bajas [4]. Al tratar el audio como una imagen (espectrograma o mel-espectrograma), se pueden aplicar arquitecturas de CNN originalmente diseñadas para visión artificial.

En resumen, la aplicación de CNNs a representaciones visuales del audio como los espectrogramas ha permitido que los modelos aprendan características discriminativas de forma automática, superando a menudo las técnicas basadas en características manuales y alcanzando niveles de precisión significativamente más altos en la clasificación de géneros musicales.

3.5. Redes Generativas y Técnicas de Deep Learning

Los modelos generativos, como las redes adversarias generativas (Generative Adversarial Networks, GAN), han demostrado ser herramientas útiles para mejorar la clasificación de géneros musicales. Las GAN pueden generar muestras sintéticas que imitan audios reales, ampliando así los conjuntos de datos de entrenamiento y reduciendo problemas de sobreajuste [14].

El enfoque propuesto por Dwivedi and Islam [14] combina GAN con transformadas matemáticas como la transformada de Fourier y la transformada Wavelet para extraer características tanto del dominio de frecuencia como del dominio temporal [14]. Esto permite capturar patrones específicos de los géneros, como la presencia de instrumentos característicos o estructuras rítmicas.

Además, la combinación de GAN con autoencoders permite reducir el ruido en los datos y mejorar la robustez de los modelos. Este enfoque ha demostrado ser particularmente efectivo en escenarios con datos limitados [16].

3.6. Optimizadores y Arquitecturas

El uso de optimizadores avanzados, como Adam, RMSProp y NADAM, ha tenido un impacto significativo en la mejora del rendimiento de los modelos en la clasificación de géneros musicales:

- **ADAM (Adaptive Moment Estimation):** Es un algoritmo de optimización basado en el cálculo de momentos (promedio y varianza) de los gradientes. Combina las ventajas de algoritmos como AdaGrad (Adaptive Gradient Algorithm) y RMSProp (Root Mean Square Propagation), adaptando la tasa de aprendizaje

para cada parámetro de manera individual. Esto le permite encontrar soluciones más rápidamente y de manera más estable, incluso en problemas que involucran grandes cantidades de datos. [28].

- **RMSProp (Root Mean Square Propagation):** Ajusta cómo se realizan los cambios en los parámetros del modelo, tomando en cuenta los gradientes anteriores. Esto ayuda a que el proceso de optimización sea más estable, especialmente cuando los gradientes tienen fluctuaciones o cambios impredecibles. [22].
- **NADAM (Nesterov-accelerated Adaptive Moment Estimation):** NADAM es una versión mejorada de ADAM que incorpora el método de aceleración de Nesterov (explicado en el apartado 2.9.1). Mientras que ADAM ajusta la tasa de aprendizaje de cada parámetro utilizando el promedio y la varianza de los gradientes, NADAM lo hace de manera más inteligente, anticipando hacia dónde se moverán los gradientes en el siguiente paso. Esta anticipación hace que las actualizaciones sean más precisas, lo que permite que el algoritmo encuentre la mejor solución en menos tiempo. Al combinar la flexibilidad de Adam con la predicción de Nesterov, NADAM puede optimizar el modelo más rápidamente en ciertos problemas, especialmente cuando los gradientes cambian mucho o son complicados [13].

Bala Ganesh et al. [4], por ejemplo, emplea NADAM para ajustar los pesos de la red y logra una precisión del 87 % al trabajar con fragmentos de 3 segundos. Por otro lado, también arquitecturas como ResNet y DenseNet se han utilizado exitosamente para analizar espectrogramas:

- **ResNet:** ResNet (Red Residual) utiliza conexiones residuales para mitigar el problema del desvanecimiento del gradiente, permitiendo entrenar redes muy profundas. Estas conexiones facilitan la propagación de la información a través de la red, lo que resulta en una mayor capacidad para extraer características relevantes de los datos [20].
- **DenseNet:** DenseNet (Red Densamente Conectada) conecta cada capa a todas las capas posteriores, lo que fomenta la reutilización de características y reduce significativamente el número de parámetros [23]. Esto facilita una mejor generalización del modelo al analizar espectrogramas complejos.

Estas arquitecturas permiten extraer características jerárquicas, lo que mejora la capacidad del modelo para generalizar a nuevos datos y contribuye a la robustez en la clasificación de géneros musicales [36, 4].

3.7. Antecedentes

En la sección 3.2 he tratado los enfoques tradicionales, pero actualmente se priorizan modelos que aprenden representaciones directamente del audio, reduciendo la intervención manual. Un ejemplo destacado es el de Nam et al. [35], cuyo modelo en dos etapas primero proyecta patrones espectrales en un espacio de alta dimensionalidad y luego utiliza deep learning no supervisado para inicializar una red neuronal que se

entrena posteriormente de forma supervisada, logrando buenos resultados en el dataset MagnaTagATune.

Paralelamente, Dieleman and Schrauwen [11] han realizado aportes fundamentales en el aprendizaje de características directamente desde el audio. En su trabajo, investigan la posibilidad de entrenar redes neuronales convolucionales directamente sobre la señal de audio cruda, sin depender de representaciones intermedias como espectrogramas o MFCCs. Sus resultados muestran que las redes pueden descubrir de manera autónoma descomposiciones frecuenciales y representaciones invariantes de fase y traslación, abriendo la puerta a sistemas verdaderamente “end-to-end”.

Otro trabajo relevante es el de Van Den Oord et al. [45], donde exploran el aprendizaje por transferencia. Su propuesta consiste en preentrenar redes profundas de forma supervisada en una tarea relacionada y transferir ese conocimiento a la tarea objetivo de clasificación musical. Este enfoque demostró que el aprendizaje por transferencia puede mejorar la generalización y el rendimiento, especialmente cuando los datos de entrenamiento son limitados.

En el artículo de Dieleman and Schrauwen [10] se desarrollan y comparan tres estrategias para el aprendizaje de características multiescala. Demuestran que incorporar información de diferentes escalas temporales mejora el rendimiento en tareas de autoetiquetado y aprendizaje de similitud, ya que distintos tipos de etiquetas (géneros, instrumentos, estados de ánimo) pueden depender de patrones presentes en diferentes escalas temporales.

Hamel et al. [18] profundiza en la importancia del pooling temporal y el aprendizaje multiescala para capturar la estructura jerárquica de la música. Sus experimentos muestran que el uso de arquitecturas capaces de agregar información a lo largo de distintas escalas temporales mejora tanto la anotación automática como el ranking de música, sentando las bases para el desarrollo de modelos más robustos y versátiles.

Los resultados de todos estos trabajos muestran valores de AUC en un rango muy próximo (entre 0.88 y 0.89), destacando la consistencia y madurez de las técnicas actuales para el autoetiquetado musical en este conjunto de datos.

3.8. Conclusiones

El análisis del estado del arte en la clasificación automática de géneros musicales revela una evolución significativa desde los métodos tradicionales basados en la extracción manual de características hasta el uso de arquitecturas profundas y técnicas generativas avanzadas. Estas innovaciones han permitido mejorar la precisión y robustez de los sistemas, facilitando aplicaciones en recomendación, organización de grandes catálogos musicales y análisis cultural.

Un hallazgo especialmente relevante es la eficacia de la segmentación en fragmentos cortos de audio, concretamente de 3 segundos, que ha demostrado mejorar el rendimiento de los modelos al permitirles capturar patrones locales más representativos y reducir la influencia de variaciones irrelevantes presentes en fragmentos largos. Esta estrategia, validada en datasets como *GTZAN*, podría aplicarse al conjunto de datos

MagnaTagATune, donde hasta ahora la mayoría de los enfoques han trabajado a nivel de clip completo. Implementar la segmentación en *MagnaTagATune* permitiría aumentar el número de muestras de entrenamiento, diversificar los patrones aprendidos y, potencialmente, mejorar la capacidad de generalización de los modelos, especialmente en un contexto multilabel.

4. Estudio previo

4.1. Introducción

Este apartado recoge los objetivos planteados, la metodología seguida y el entorno tecnológico utilizado, así como la documentación consultada y los datos empleados. También se detalla la planificación inicial y real del trabajo, junto con una estimación presupuestaria.

4.2. Objetivos

Este Trabajo de Fin de Máster tiene como objetivo diseñar e implementar un sistema de clasificación de géneros musicales basado en redes neuronales convolucionales (Convolutional Neural Networks, CNNs [2.6]), optimizando el almacenamiento y procesamiento de los espectrogramas generados a partir de los fragmentos de audio.

Para ello, se utilizará el conjunto de datos *MagnaTagATune*, que contiene una amplia variedad de fragmentos musicales etiquetados con distintos géneros y características. El desarrollo de este proyecto incluirá la preparación y preprocesamiento de los audios, la implementación del modelo de clasificación y la evaluación de su desempeño mediante métricas específicas.

A continuación, se presentan los principales objetivos del estudio:

1. Diseñar y desarrollar un sistema de clasificación automática de géneros musicales basado en redes neuronales convolucionales.
2. Implementar un proceso de preprocesamiento que segmente y transforme los audios en espectrogramas.
3. Analizar el rendimiento del modelo utilizando métricas para la clasificación de audio.

4.3. Metodología

En este apartado se describe la metodología seguida para el desarrollo del sistema de clasificación automática de géneros musicales. Se detallan el conjunto de datos utilizado, las decisiones relativas a su almacenamiento y preprocesamiento, la implementación del modelo, los algoritmos de aprendizaje empleados, así como los procedimientos de entrenamiento y evaluación. Finalmente, se explican las herramientas utilizadas para la documentación y gestión del proyecto.

4.3.1. Documentación

La documentación del proyecto se redactará y organizará utilizando *Overleaf*, una plataforma colaborativa basada en *LaTeX* que facilita la creación de documentos técnicos de forma profesional. Esta herramienta permitirá estructurar el informe, integrar gráficos y tablas, y mantener un control de versiones efectivo del texto. Además, se utilizará *ChatGPT-4* como apoyo para mejorar la formalidad y claridad en la redacción, así como para resolver dudas puntuales relacionadas con la implementación y depuración del código. El uso de control de versiones mediante *Git* y, además, el uso de *GitHub* permitirá mantener un historial detallado tanto del código como de la documentación, garantizando trazabilidad y reproducibilidad.

4.3.2. Dataset Utilizado en el Estudio

El dataset utilizado se titula *MagnaTagATune*¹ y es una base de datos que se utiliza frecuentemente en tareas de clasificación musical, como por ejemplo en los artículos de Kim et al. [27] y Won et al. [46]. El objetivo inicial de este dataset era proporcionar anotaciones detalladas realizadas por humanos, convirtiéndolo en un recurso valioso para tareas de aprendizaje automático en el ámbito del procesamiento de audio.

Descripción del Dataset

El conjunto de datos *MagnaTagATune* [29] está compuesto por un total de 25,863 clips de audio, cada uno con una duración de 29 segundos. Estos fragmentos han sido extraídos de 5,223 canciones y abarcan una amplia variedad de géneros musicales, incluyendo clásica, New Age, electrónica, rock, pop, world, jazz, blues, metal y punk.

Cada clip de audio está acompañado de un vector de anotaciones binarias correspondientes a 188 etiquetas, las cuales fueron generadas a partir de las contribuciones de los jugadores del juego *TagATune*. Estas etiquetas pueden incluir información sobre géneros, instrumentos, estado de ánimo y otras características relevantes para la identificación y clasificación musical.

Aunque intuitivamente esperaríamos que todos los fragmentos de audio pertenecientes a una misma canción compartieran las mismas etiquetas de género, en este dataset no es así. Debido a que las clasificaciones fueron realizadas por diferentes personas en distintos momentos, se observan inconsistencias en las etiquetas asignadas a los fragmentos.

Los archivos de audio están en formato MP3, codificados a 32kbps y 16kHz, lo que resulta en un tamaño total aproximado de 3GB para el conjunto completo de datos.

¹Esta base de datos es accesible desde <https://mirg.city.ac.uk/datasets/magnatagatune/>

Estructura y Organización del Dataset

Los datos en *MagnaTagATune* están organizados en archivos de audio y archivos con distintas anotaciones. Los archivos de audio están almacenados en formato MP3, mientras que las etiquetas se presentan en un archivo separado en formato CSV, donde cada fila corresponde a un clip de audio y contiene su identificador junto con las etiquetas binarias asignadas.

Los archivos de audio están organizados en distintas carpetas con el propósito de facilitar, en etapas posteriores, la división del dataset para el entrenamiento y la evaluación de algoritmos, usando algunas carpetas como conjunto de entrenamiento (train) y otras como conjunto de prueba (test). Sin embargo, en nuestro caso, al generar espectrogramas a partir de estos audios, cada carpeta contendrá una gran cantidad de datos derivados, lo que nos obligará a realizar una nueva división de la información para asegurar una correcta separación entre los conjuntos de entrenamiento y prueba.

Motivación para la Selección del Dataset

El dataset *MagnaTagATune* ha sido elegido para este trabajo debido a:

- **Variedad de anotaciones:** La multitud de etiquetas detalladas (concretamente 188), proporcionan una base sólida para realizar tareas de clasificación, específicamente de géneros musicales.
- **Variedad de géneros:** La presencia de gran variedad de géneros permite evaluar la capacidad del modelo para diferenciar entre distintas categorías musicales.
- **Uso extendido en la comunidad científica:** Este dataset ha sido empleado en numerosos estudios previos, lo que facilita la comparación de resultados con investigaciones existentes.
- **Tamaño y accesibilidad:** A pesar de su riqueza en datos, el conjunto de datos es manejable en términos de almacenamiento y procesamiento, permitiendo su uso en un entorno de desarrollo sin requerimientos computacionales excesivos.

4.3.3. Gestión del almacenamiento de los espectrogramas

Una de las principales dificultades de este proyecto es que plantea un desafío en términos de almacenamiento y acceso eficiente a los datos, ya que existe un elevado número de espectrogramas generados (en torno a 300,000) que deben ser de rápido acceso. Para abordar este problema, he evaluado diversas opciones, teniendo en cuenta factores como facilidad de acceso, compatibilidad y eficiencia en la carga de datos durante el entrenamiento del modelo.

Las opciones consideradas han sido:

- **Almacenamiento en plataformas en la nube (*Google Drive*, *Dropbox*, *OneDrive* y *Google Cloud Storage*).** Estas opciones ofrecen accesibilidad remota y escalabilidad, pero pueden introducir latencias en el acceso a los datos

y requieren una conexión a internet constante, lo que puede ralentizar la consulta de los espectrogramas.

- **Almacenamiento en formatos comprimidos (.hdf5, .npz).** Estos formatos permiten almacenar múltiples espectrogramas en un único archivo, facilitando el acceso y reduciendo el tamaño de almacenamiento, pero son lentos a la hora de recuperar datos específicos.
- **Bases de datos locales (SQLite).** Esta opción permite almacenar los espectrogramas de manera estructurada y eficiente, facilitando la indexación y el acceso a los datos sin necesidad de depender de la nube.
- **Reducción de la cantidad de datos.** Una alternativa es seleccionar un subconjunto representativo del conjunto de datos original, aunque esto podría reducir la capacidad del modelo para generalizar sobre una amplia variedad de géneros.

Tras evaluar los pros y los contras de cada opción, he decidido utilizar *SQLite* como base de datos local para almacenar los espectrogramas. Esta elección se basa en varios factores clave:

- Permite acceder a los datos de manera estructurada y eficiente sin necesidad de conexión a internet.
- Optimiza el almacenamiento, ya que los espectrogramas pueden guardarse en formato binario dentro de la base de datos.
- Facilita la consulta y recuperación de datos específicos durante el entrenamiento del modelo.
- Es compatible con Python y las bibliotecas que planeo utilizar en el proyecto como *TensorFlow* y *PyTorch*.

4.3.4. Implementación

La implementación se desarrollará con *Python*, utilizando *Visual Studio Code* como entorno principal. El preprocesamiento de audio se realizará con *pydub* para la conversión y recorte, y *librosa* para la extracción de características y generación de mel-espectrogramas. El modelo de clasificación se construirá y entrenará con *TensorFlow/Keras*. El control de versiones y la gestión del código fuente se realizarán con *Git*, mientras que la planificación y seguimiento de tareas se gestionarán con *Trello* y *Clockify*. La integración de todas estas herramientas permitirá un flujo de trabajo eficiente, reproducible y colaborativo.

4.3.5. Elección de algoritmos

Para la tarea de clasificación automática de géneros musicales, se utilizarán redes neuronales convolucionales (CNN [2.6](#)), dada su eficacia en el procesamiento de datos visuales como los espectrogramas. Se priorizará la facilidad de uso y la capacidad del modelo para aprender patrones temporales complejos. El preprocesamiento incluirá la

fragmentación de los audios en segmentos de duración fija y el uso de ventanas deslizantes, con el objetivo de incrementar el número de muestras y mejorar la capacidad de generalización del modelo.

4.3.6. Entrenamiento y evaluación

En el entrenamiento del modelo se utilizarán mel-espectrogramas generados a partir de los archivos de audio preprocesados. Para asegurar la robustez de los resultados y evitar la dependencia de una única partición de los datos, se empleará validación cruzada con *k-folds* (*K-Fold Cross Validation*). Durante el entrenamiento, se aplicará la técnica de *early stopping* para evitar el sobreajuste. Las métricas utilizadas para evaluar el rendimiento incluirán *accuracy*, *precision*, *recall*, *AUC* y *F1-score*, proporcionando así una visión integral del comportamiento del modelo, especialmente relevante en tareas multilabel.

4.4. Planificación

El desarrollo de este Trabajo Fin de Máster se ha llevado a cabo siguiendo un ciclo ágil iterativo, en el cual la programación y la redacción del documento se han realizado de forma paralela. De esta forma he podido ir mejorando progresivamente tanto la implementación técnica como la teórica, ya que los avances en el código influían directamente en la necesidad de adaptar y ampliar secciones del documento, y viceversa.

4.4.1. Metodología de Trabajo

Al tratarse de un proyecto individual, se ha optado por un enfoque de desarrollo ágil e iterativo, adaptado a las necesidades y características del trabajo. A continuación, se describen las principales decisiones organizativas adoptadas:

- **Planificación:** Se ha organizado el trabajo en iteraciones de aproximadamente dos semanas, lo que ha facilitado una revisión periódica del progreso.
- **Revisiones periódicas:** Cada dos semanas se han llevado a cabo reuniones de seguimiento con mi tutor, donde se ha evaluado el estado del proyecto y se han definido los siguientes objetivos.
- **Seguimiento diario:** Al no contar con un equipo, no se han realizado reuniones diarias. No obstante, cada día se ha llevado a cabo una revisión personal del trabajo pendiente y los objetivos a corto plazo.
- **Gestión de tareas:** Se ha utilizado Trello como herramienta principal para la planificación y el control del progreso. En ella se ha documentado el estado de las tareas, bloqueos detectados y soluciones aplicadas, especialmente en lo referente al entorno de desarrollo con el fin de agilizar la resolución si se volviera a producir.

- **Registro del tiempo:** Se ha empleado Clockify para contabilizar las horas dedicadas a cada fase del proyecto, lo cual ha permitido analizar la distribución del tiempo y ajustar la planificación según las necesidades reales.

4.4.2. Planificación y fases del proyecto inicial

A continuación se presenta la planificación inicial para este TFM:

Fase	Periodo	Horas estimadas
Formación y definición	dic 2024 – ene 2025	40 h
Estado del arte y revisión bibliográfica	dic 2024 – ene 2025	40 h
Diseño y planificación técnica	ene 2025	25 h
Entorno y análisis de datos	ene – feb 2025	35 h
Espectrogramas y entrenamiento	feb – mar 2025	55 h
Evaluación y validación	mar – abr 2025	35 h
Redacción de la memoria	ene – abr 2025	50 h
Revisión final y defensa	abr – may 2025	20 h
Total		300 h

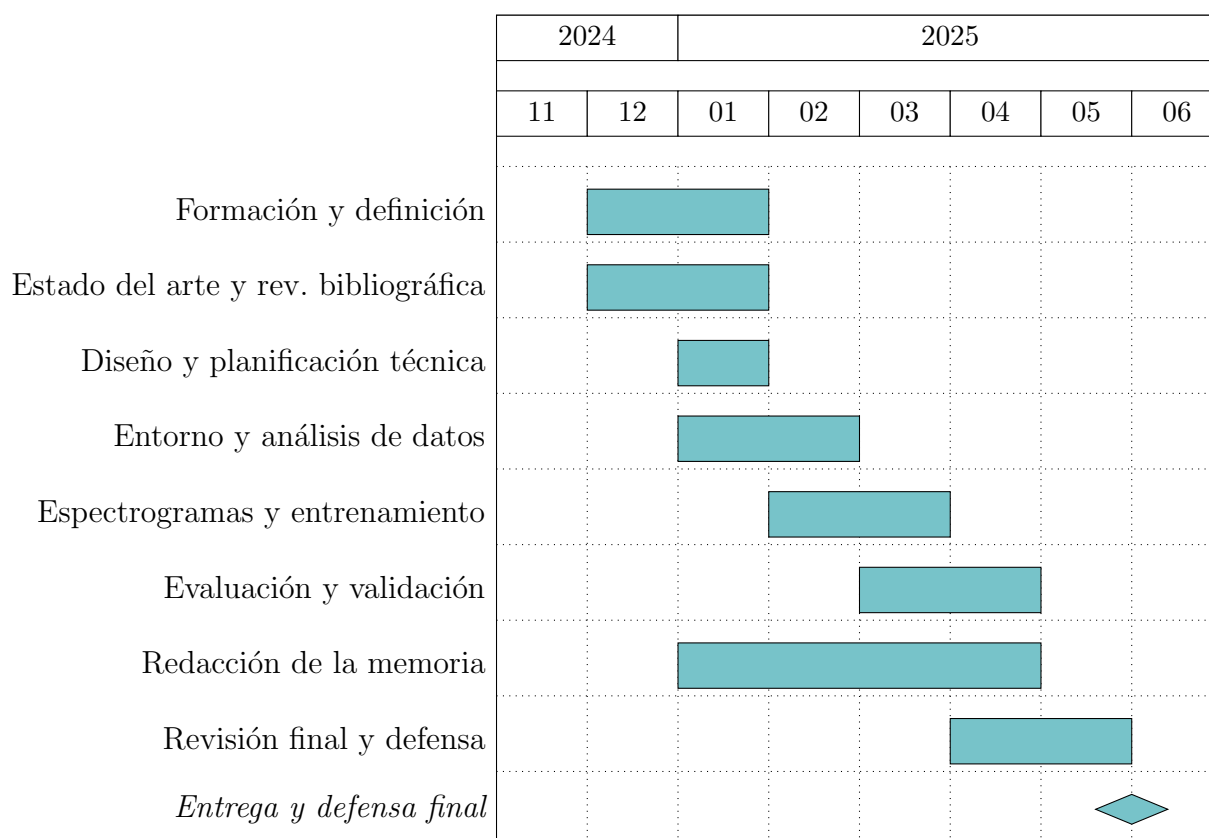
Cuadro 4.1: Distribución estimada de horas por fase del TFM

1. **Fase de formación y definición:** Durante las primeras semanas del proyecto, se dedicará tiempo a la adquisición de conocimientos clave sobre redes neuronales convolucionales (CNN), procesamiento de audio y generación de espectrogramas. Paralelamente, se definirá de forma precisa el problema a abordar, los objetivos del TFM y los criterios de éxito del modelo.
2. **Estado del arte y revisión bibliográfica:** Se realizará una revisión exhaustiva de la literatura existente, tanto en el ámbito del reconocimiento musical como en técnicas aplicadas con deep learning. Este estudio permitirá identificar datasets comúnmente utilizados y posibles áreas de mejora.
3. **Diseño y planificación técnica:** Una vez asentadas las bases teóricas, se definirá el entorno tecnológico, las herramientas de desarrollo, los recursos de computación y la estructura general del sistema a implementar. También se planificará la arquitectura del modelo.
4. **Preparación del entorno y análisis de datos:** Se procederá a la instalación y configuración de las librerías necesarias y se realizará un análisis exploratorio del dataset elegido. Se definirán las variables relevantes y se estudiará la viabilidad del dataset respecto a los objetivos del proyecto.
5. **Generación de espectrogramas, desarrollo y entrenamiento de modelos:** A partir de los archivos de audio del dataset, se generarán espectrogramas. Con los datos ya procesados, se entrenará el modelo de clasificación.
6. **Evaluación y validación del sistema:** Una vez obtenidos los modelos entrenados, se evaluará su rendimiento utilizando métricas adecuadas como la precisión,

recall o F1-score.

7. **Redacción y elaboración de la memoria:** En paralelo al desarrollo técnico, se redactará de forma progresiva la memoria del TFM. Esta incluirá tanto los fundamentos teóricos como las decisiones de diseño, los resultados obtenidos y las conclusiones derivadas del proyecto.
8. **Revisión final y defensa:** Finalmente, se revisará la documentación, se preparará la presentación para la defensa y se realizarán los últimos ajustes en base a las recomendaciones del tutor o revisores.

Si representamos con un diagrama de Gantt:



4.4.3. Planificación y fases del proyecto real

El desarrollo del TFM ha seguido una estrategia iterativa y ágil, donde la programación y la elaboración de la memoria han evolucionado de forma conjunta. A continuación, se describen las fases clave del proyecto, no como bloques rígidos, sino como procesos entrelazados que se han ido mejorando y modificando de manera continua:

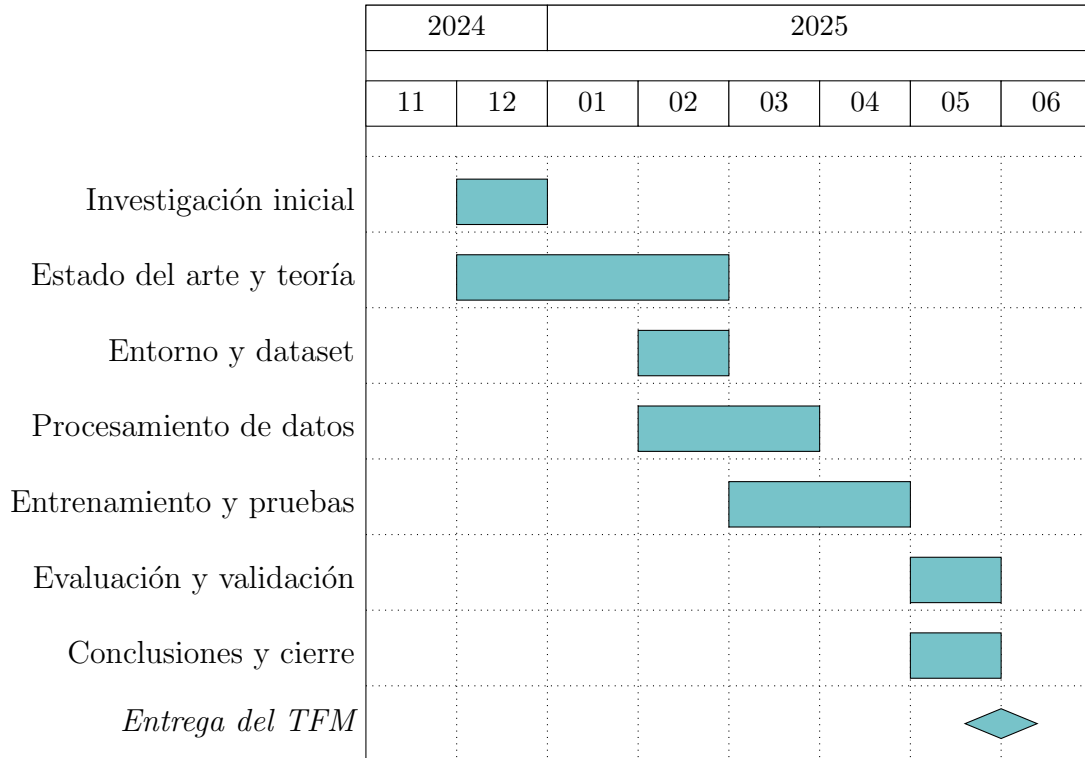
Fase	Periodo	Horas estimadas
Formación previa y definición	dic 2024 – ene 2025	18,2 h
Estado del arte y revisión bibliográfica	dic 2024 – ene 2025	45,7 h
Diseño y planificación técnica	ene 2025	12,4 h
Entorno y análisis de datos	ene – feb 2025	32,8 h
Espectrogramas y entrenamiento	feb – mar 2025	57,1 h
Evaluación y validación	mar – may 2025	38,4 h
Redacción de la memoria	ene – jun 2025	83,5 h
Revisión final y defensa	may – jun 2025	18,2 h
Total		306,7 h

Cuadro 4.2: Distribución estimada de horas por fase del TFM

1. **Investigación inicial:** Durante las primeras semanas, dediqué tiempo a comprender los conceptos fundamentales necesarios para abordar el trabajo, como las redes neuronales convolucionales (CNN) y el uso de espectrogramas para representar señales de audio. Esta etapa incluyó la visualización de vídeos formativos, lecturas de documentación técnica y pruebas en el entorno de desarrollo.
2. **Documentación para el estado del arte y fundamentos teóricos:** La redacción del estado del arte y los fundamentos teóricos se desarrollaron de forma paralela a la etapa anterior. A medida que analizaba trabajos relacionados, surgía la necesidad de incorporar explicaciones más detalladas de ciertos conceptos técnicos. Este enfoque me permitió construir un marco teórico coherente y bien fundamentado, ajustado a los requerimientos prácticos del proyecto.
3. **Preparación del entorno y análisis del dataset:** En esta fase configuré el entorno tecnológico necesario (Python, librerías de deep learning, entorno de GPU, etc.) y realicé un análisis exploratorio del dataset MagnaTagATune. Este análisis fue clave para entender la distribución de los géneros musicales, la organización de los datos y los posibles enfoques de clasificación. Al mismo tiempo, definí las estrategias de validación y segmentación, identificando la posibilidad de aplicar validación cruzada.
4. **Procesamiento de datos y generación de espectrogramas:** Posteriormente, se implementaron las herramientas necesarias para convertir los clips de audio en espectrogramas. Este proceso generó una gran cantidad de datos, lo que supuso un reto a nivel de almacenamiento. Tras evaluar distintas opciones, decidí utilizar una base de datos local (SQLite).
5. **Entrenamiento de modelos y experimentación:** Una vez generados los datos, comencé a entrenar distintos modelos de clasificación, ajustando hiperparámetros y probando distintas configuraciones. Esta fase fue muy iterativa, con numerosos ciclos de prueba y error, evaluación de resultados y reentrenamiento.
6. **Evaluación de resultados y validación:** Con los modelos entrenados, se procedió a su evaluación mediante métricas estándar. Se analizaron los resultados obtenidos, se identificaron posibles sesgos y se revisó la capacidad del modelo para generalizar frente a nuevos datos.

7. **Conclusiones y cierre del proyecto:** Finalmente, se extrajeron las conclusiones más relevantes del trabajo, reflexionando sobre las limitaciones del enfoque propuesto y señalando posibles líneas de mejora y ampliación del estudio.

A continuación, se presenta un diagrama de Gantt que ilustra visualmente la planificación:



4.5. Presupuesto

Concepto	Detalle	Coste estimado (€)
Horas de trabajo	306,7h × 20€/h	6.134,00
Consumo eléctrico	306,7h × 0,13€/kWh × 0,15kW	5,98
Amortización de hardware	Ordenador de mesa gama media-alta (3 años)	400,00
Suscripción a GitHub Pro	12€/mes × 6 meses	72,00
Conexión a Internet	Parte proporcional de 6 meses de tarifa (50€/mes)	150,00
Total estimado		6.761,98

Cuadro 4.3: Presupuesto estimado para el desarrollo del TFM

Para la elaboración de este presupuesto se han tenido en cuenta diversos factores que representan costes directos e indirectos asociados. El precio de las horas de trabajo

se ha obtenido a través de los sueldos medios registrados para ingenieros de software en España por Talent[43] y Glassdoor[15]².

En cuanto al consumo eléctrico, se ha registrado el consumo aproximado del equipo durante las 306,7 horas de trabajo, con un consumo estimado de 0,1 kW (ordenador de sobremesa de gama media-alta) y una tarifa media de 0,13 €/kWh, resultando en un coste muy reducido.

La amortización del hardware se calcula en función de su coste total dividido entre el periodo total de amortización (en meses), y luego se multiplica por la duración del proyecto:

$$\text{Coste imputado} = \left(\frac{1,200 \text{ €}}{36} \right) \times 12 = 33,33 \text{ €} \times 12 = 400 \text{ €}$$

Además, en el presupuesto se han incluido gastos derivados de herramientas y servicios como GitHub Pro (12€/mes). Por último, se ha incluido la proporción correspondiente de la tarifa mensual de conexión a Internet.

²Talent y Glassdoor son plataformas de búsqueda de empleo que publican estadísticas salariales basadas en datos de usuarios y ofertas activas.

5. Implementación

5.1. Introducción

En este capítulo explicaremos el proceso de investigación interna del dataset y la posterior implementación de nuestras redes convolucionales.

5.2. Análisis exploratorio de los datos

Con el objetivo de identificar posibles relaciones entre los géneros musicales presentes en el conjunto de datos, se ha llevado a cabo un análisis de correlación. Esta técnica estadística nos permite cuantificar la intensidad y dirección de la relación entre dos variables, en este caso, distintos géneros musicales.

Se parte de la premisa de que algunos géneros musicales podrían presentar características comunes, lo que daría lugar a una correlación positiva entre ellos. A continuación, se presentan dos tablas: la primera tabla 5.1 representa las correlaciones más altas encontradas entre géneros, mientras que la segunda 5.2 muestra a aquellos cuya correlación es prácticamente nula, lo cual significa que entre ellos no existe ninguna relación significativa.

Genero_1	Genero_2	Correlacion
hard rock	metal	0.534015
hip hop	rap	0.525677
electronic	techno	0.523670
jazz	jazzy	0.517258
heavy metal	metal	0.413903
indian	eastern	0.407724
hard rock	heavy metal	0.387497

Cuadro 5.1: Correlaciones más altas entre géneros musicales

Genero_1	Genero_2	Correlacion
orchestra	dark	-0.000228
pop	strange	-0.000221
baroque	female opera	0.000152
jazz	electro	-0.000102
blues	pop	0.000088
orchestra	medieval	0.000004
oriental	strange	-0.000002

Cuadro 5.2: Correlaciones más bajas entre géneros musicales

Este análisis preliminar nos ha ayudado a ver la existencia de ciertas similitudes entre géneros y, por otro lado, también confirma que muchos estilos musicales presentan una independencia casi total entre sí.

5.3. Criterios de clasificación de géneros musicales

Una vez se ha analizado la correlación entre las etiquetas, planteamos reorganizarlas con el fin de agrupar aquellas que presentan similitudes claras. Esta reestructuración permitiría reducir la redundancia entre categorías y facilitaría que el modelo generalice mejor durante el entrenamiento. En definitiva, esta nueva organización de etiquetas contribuirá a mejorar tanto la coherencia del conjunto de datos como la precisión en la clasificación.

Se han establecido cinco grandes grupos que responden a criterios de afinidad o contexto cultural comunes en la categorización musical:

1. Géneros con una fuerte identidad propia: estos géneros están claramente diferenciados y sus características son lo suficientemente distintivas como para ser tratadas como clases individuales. Estos géneros son: **Punk, Blues, Pop, Country** y **Reggae**.

2. Géneros derivados de una misma familia musical: Dentro de este grupo agrupamos los subgéneros que comparten rasgos comunes y poseen o no orígenes similares:

- **Rock:** hard rock, soft rock
- **Metal:** heavy metal, metal
- **Jazz:** jazz, jazzy
- **Electronic:** techno, trance, house, disco, industrial
- **Funk:** funk, funky
- **Hip Hop:** hip hop, rap

3. Música basada en instrumentos acústicos o tradiciones: Géneros que utilizan instrumentos acústicos y estructuras tradicionales:

- **Folk acústico:** folk, celtic, irish
- **Folk antiguo:** medieval, tribal

4. Música tradicional agrupada por región geográfica: Agrupaciones de géneros musicales según su origen cultural y características propias:

- **Middle Eastern:** middle eastern, arabic
- **Indian:** indian, india

- **Oriental:** eastern, oriental
- **Spanish:** spanish

5. Música ambiental y clásica: Géneros con un enfoque atmosférico o de tradición clásica:

- **Classical:** classical, opera
- **Ambient:** new age, space, eerie, drone

Esta nueva clasificación ayudará al modelo a identificar patrones con mayor claridad y a mejorar la diferenciación entre clases, al trabajar con categorías más consistentes y menos redundantes.

5.4. Visualización

Para comprender mejor los datos con los que vamos a trabajar, vamos a aplicar diversas técnicas de visualización dentro del análisis exploratorio. Esta etapa resulta fundamental antes de abordar cualquier tipo de modelado, ya que permite detectar posibles sesgos y desbalances.

En primer lugar, la gráfica [5.1](#) muestra la distribución de canciones por género. Este gráfico de barras nos representa de manera muy clara la desigualdad en la frecuencia de aparición entre los distintos géneros musicales. Además, se observa una fuerte presencia de géneros como *classical*, *electronic* y *ambient*, los cuales superan ampliamente en número al resto. Este desbalance es importante tenerlo en cuenta, ya que puede influir negativamente en el entrenamiento del modelo, provocando que tienda a favorecer las clases mayoritarias. En consecuencia, se considerará aplicar técnicas de balanceo o ponderación de clases en fases posteriores.

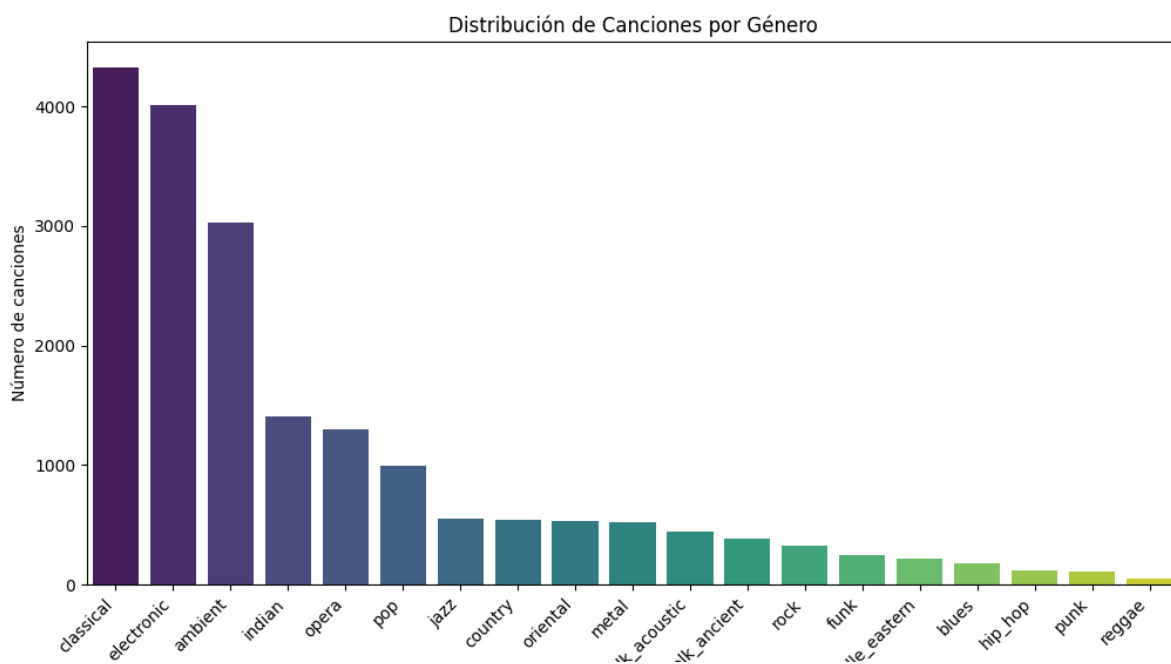


Figura 5.1: Distribución canciones por género

La visualización también permite identificar qué géneros aparecen menos, como puede ser *reggae*, *punk* o *hip hop*, que cuentan con un número significativamente menor de ejemplos.

Por otro lado, en la imagen 5.2 podemos ver representada una matriz de correlación entre géneros. A través del uso de un mapa de calor, esta imagen facilita la detección de relaciones estadísticas entre pares de etiquetas. Como es esperable, los valores más altos (coloreados en rojo) aparecen en la diagonal principal, ya que cada género está perfectamente correlacionado consigo mismo. No obstante, también pueden apreciarse ciertas correlaciones positivas relevantes entre géneros distintos. Por ejemplo, *rock* y *metal* presentan un coeficiente de correlación de 0.49, mientras que *hip hop* y *funk* también muestran cierta proximidad.

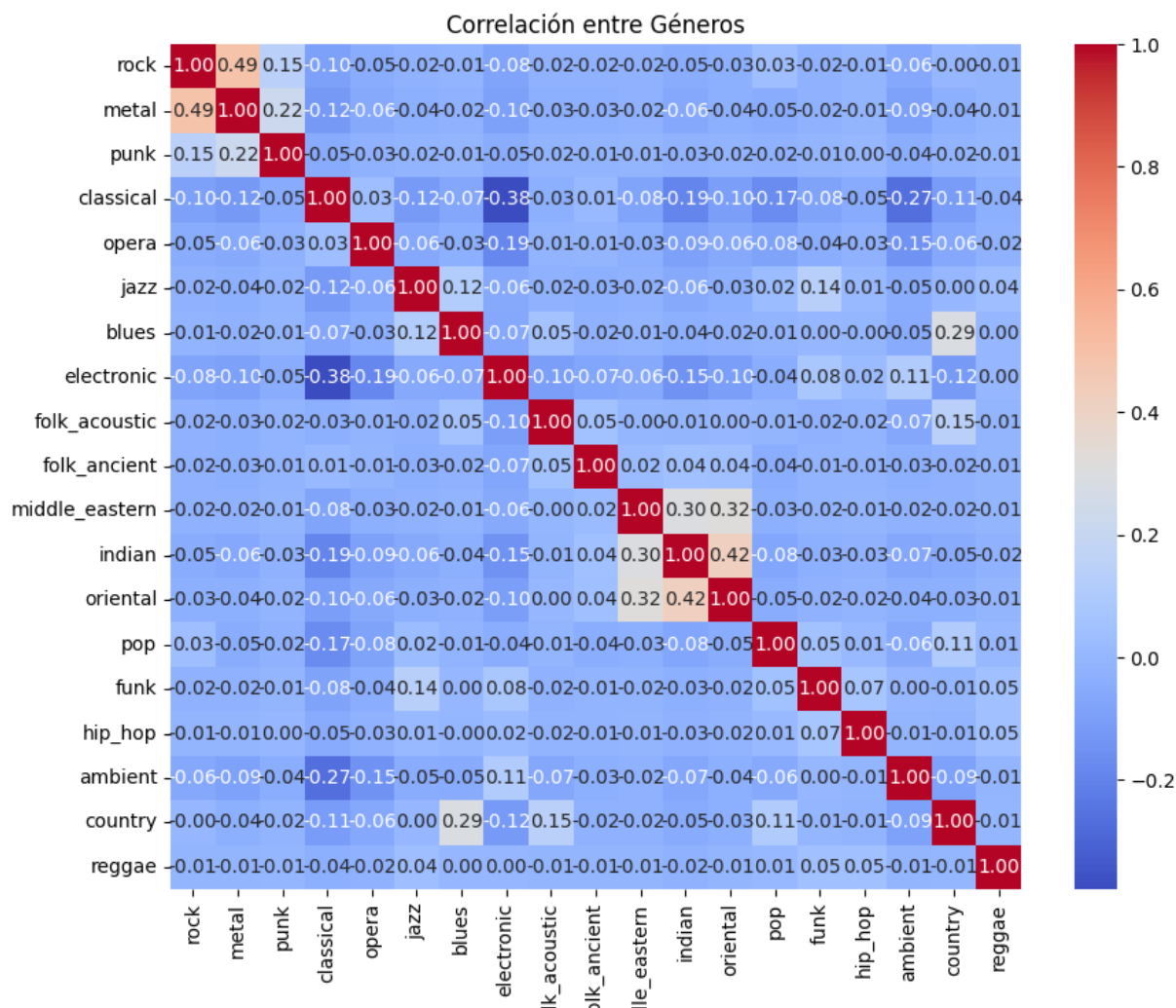


Figura 5.2: Correlación entre géneros

Estas observaciones coinciden con los resultados numéricos presentados en el apartado anterior 5.2, donde se recogen las correlaciones más altas y más bajas respectivamente. Es interesante destacar que algunos géneros, como *blues* y *pop* o *orchestra* y *medieval*, tienen una correlación prácticamente nula, lo que refuerza la idea de que muchos estilos musicales presentan patrones sonoros totalmente independientes.

La matriz de correlación también valida la estrategia explicada en la sección 5.3 para la reorganización de etiquetas.

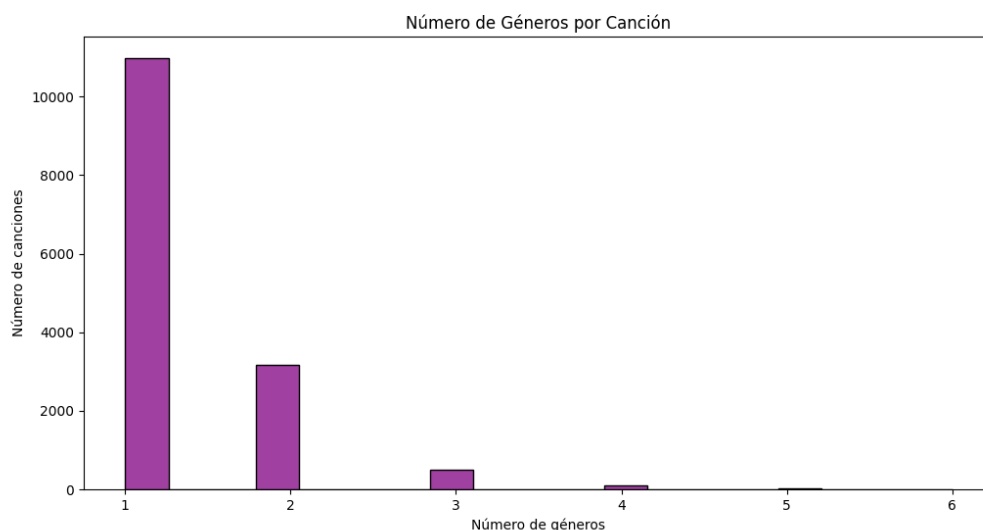


Figura 5.3: Número de géneros por canción

En la figura 5.3 se muestra el número de géneros asignados por canción, lo que nos proporciona una visión más precisa sobre la presencia de la multi-etiqueta en nuestro dataset. Como se puede observar, la gran mayoría de las canciones (más de 10.000) están etiquetadas con un solo género, lo que sugiere una fuerte tendencia hacia una clasificación unitaria en el dataset original.

El hecho de que existan canciones con múltiples géneros refuerza la necesidad de abordar este problema como una tarea de clasificación multi-etiqueta, en lugar de una clasificación tradicional. Esta distinción es clave, ya que afecta directamente a la elección de la arquitectura de red, la función de pérdida y la métrica de evaluación. Por lo tanto, esta visualización resulta especialmente útil para justificar ciertas decisiones de diseño en la implementación.

5.5. Generación de espectrogramas

Para que las redes convolucionales puedan trabajar con los audios, es necesario convertir las señales sonoras en representaciones visuales que capturen sus características espectrales y temporales. Para ello, se opta por el uso de espectrogramas, que permiten representar la intensidad de distintas frecuencias a lo largo del tiempo.

Existen distintas estrategias para generar estos espectrogramas a partir de los audios del dataset, cada una con sus ventajas y desventajas. A continuación se analizan las cuatro principales:

Opción 1: Cortar el audio en fragmentos de 3 segundos y generar un espectrograma por fragmento

Ventajas	Desventajas
Método directo y fácil de implementar.	Se pierde continuidad entre fragmentos, lo que puede afectar a la calidad de las características extraídas.
Cada espectrograma se corresponde con un fragmento de audio claramente definido.	Información relevante puede quedar en los bordes de los cortes y no estar bien representada.
Bajo coste computacional: solo se generan 10 fragmentos por audio.	Menor número de muestras por audio, lo que limita la diversidad del conjunto de entrenamiento.

Cuadro 5.3: Corte directo en fragmentos de 3 segundos

Opción 2: Generar el espectrograma completo del audio y cortarlo posteriormente

Ventajas	Desventajas
Permite un tratamiento global del audio antes de segmentar.	El espectrograma completo ocupa más memoria y es costoso de procesar.
Facilita tareas de normalización o procesamiento sobre la señal entera.	Cortar sobre la imagen puede generar fragmentos que no estén perfectamente alineados.

Cuadro 5.4: Segmentación posterior al espectrograma completo

Opción 3: Ventanas deslizantes sin solapamiento

Ventajas	Desventajas
Aumenta el número de muestras por audio.	Puede haber pérdida de información en las transiciones entre ventanas.
Permite un control claro sobre la duración de cada fragmento.	No garantiza una cobertura continua del contenido, lo que afecta a géneros con transiciones rápidas.

Cuadro 5.5: Ventanas sin solapamiento

Opción 4: Ventanas deslizantes con solapamiento

Esta fue la opción finalmente seleccionada, al ofrecer el mejor equilibrio entre cobertura temporal, diversidad de muestras y robustez para el aprendizaje del modelo.

Se aplica una ventana de 3 segundos con un solapamiento de 1.5 segundos entre cada fragmento consecutivo. Esto significa que cada nuevo espectrograma se genera desplazando la ventana 1.5 segundos respecto al anterior, garantizando así que cada instante de tiempo aparece representado en múltiples espectrogramas.

Ventajas	Desventajas
Aumenta considerablemente el número total de muestras por audio (aprox. 19 fragmentos por archivo de 30 segundos).	Mayor coste computacional en términos de procesamiento y almacenamiento.
Mejora la cobertura temporal: cada instante del audio aparece en varias ventanas.	Riesgo de sobreajuste si el modelo no generaliza bien ante muestras muy similares.
Reduce la pérdida de información en los bordes gracias al solapamiento.	
Aumenta la diversidad del conjunto de entrenamiento, mejorando el rendimiento del modelo.	

Cuadro 5.6: Ventanas con solapamiento de 1.5 segundos

Como resultado de esta configuración, se generan un total de **394 849 espectrogramas**, lo que proporciona un conjunto de entrenamiento amplio y con una representación detallada del contenido temporal de los audios.

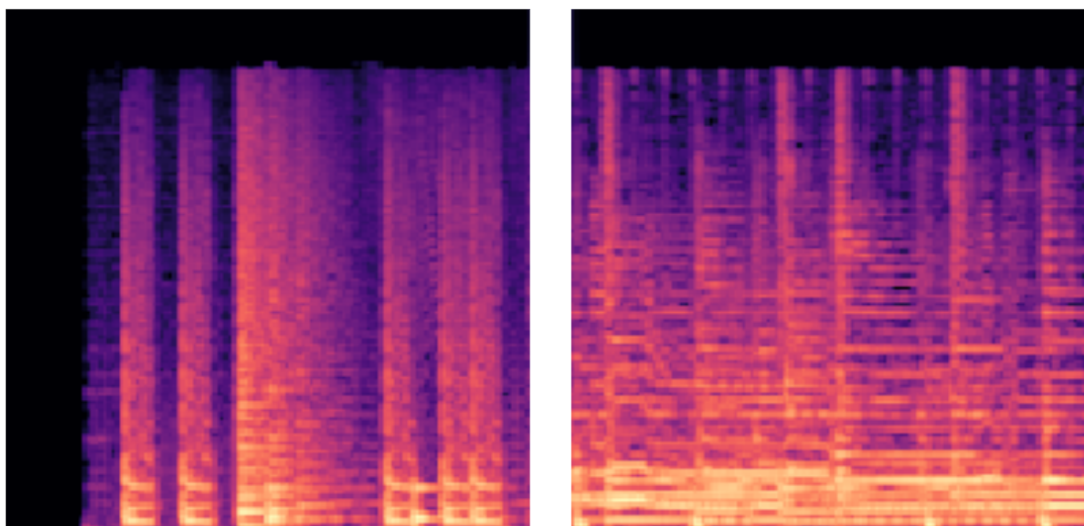


Figura 5.4: Ejemplos de espectrogramas generados

En la Figura 5.4 se pueden observar dos ejemplos reales de espectrogramas generados con lo explicado anteriormente. Estas imágenes reflejan la variación de energía a través del tiempo en distintas bandas de frecuencia, lo cual será aprovechado por las redes convolucionales para extraer patrones representativos de cada género musical.

5.6. Entrenamiento

Para comenzar con el apartado de entrenamiento, el primer paso ha sido redimensionar los espectrogramas a una resolución fija de 128x128 píxeles. Esta decisión se ha tomado con el fin de garantizar que todas las entradas tengan el mismo tamaño. Además, al escoger un tamaño relativamente pequeño, se reduce el coste computacional y el uso de memoria durante el entrenamiento, sin necesidad de perder una cantidad excesiva de información relevante.

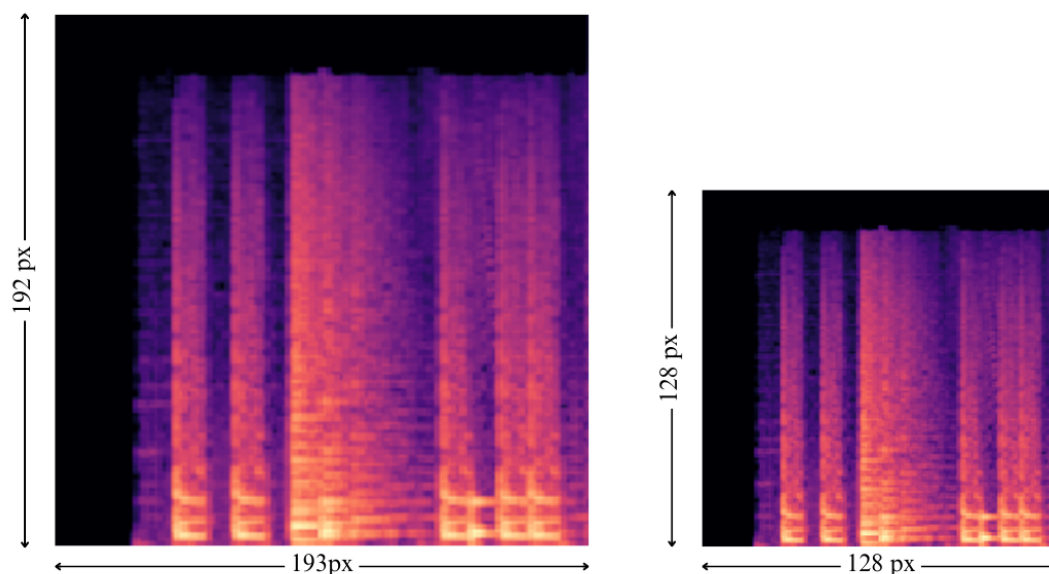


Figura 5.5: Redimensionamiento de los espectrogramas

Tras el redimensionado, se ha aplicado una normalización de los valores de píxeles (dividiendo entre 255) para llevar los valores de los píxeles, que originalmente están en el rango $[0, 255]$, al rango $[0, 1]$. Esta normalización es una práctica habitual en deep learning [26] porque permite que la red neuronal entrene de forma más estable y eficiente, evitando problemas derivados de escalas numéricas muy distintas entre distintas variables y evitando también que las redes trabajen con números muy grandes dificulten el aprendizaje.

La figura 5.6 muestra la distribución de los valores de los píxeles antes (en color morado) y después (en color amarillo) del proceso de normalización. Como puede observarse, existe una alta concentración en los valores máximos (255 en la escala original y 1.0 en la normalizada).

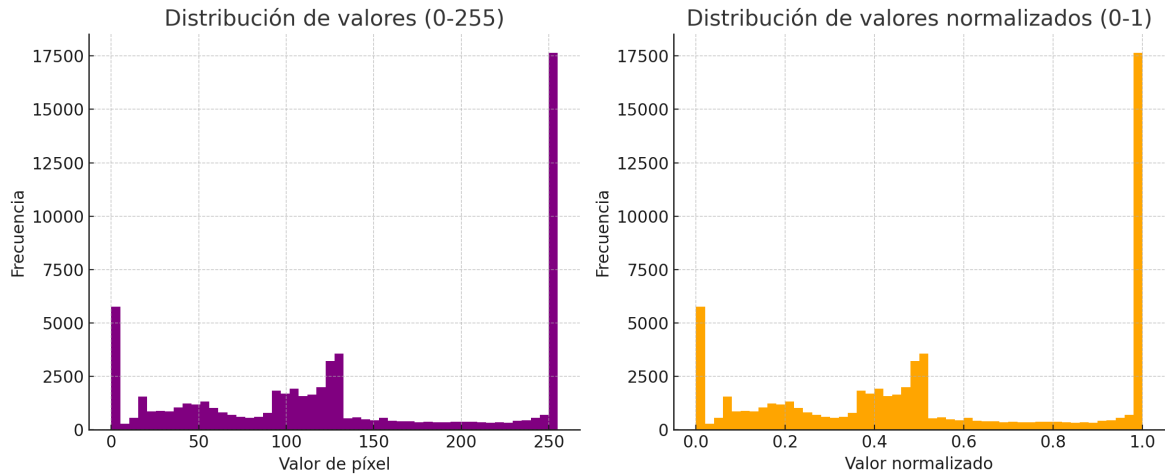


Figura 5.6: Distribución de valores de píxeles en un espectrograma

Como se puede ver en la figura 5.7, la red utiliza varias capas convolucionales Conv2D, que funcionan como filtros que recorren el espectrograma buscando patrones importantes. Después de cada una de estas capas, se ha incluido una capa MaxPooling2D, que se encarga de reducir el tamaño de las representaciones internas manteniendo la información más relevante. Esto permite que el modelo sea más rápido y consuma menos recursos, pero sin perder la información que ha aprendido.

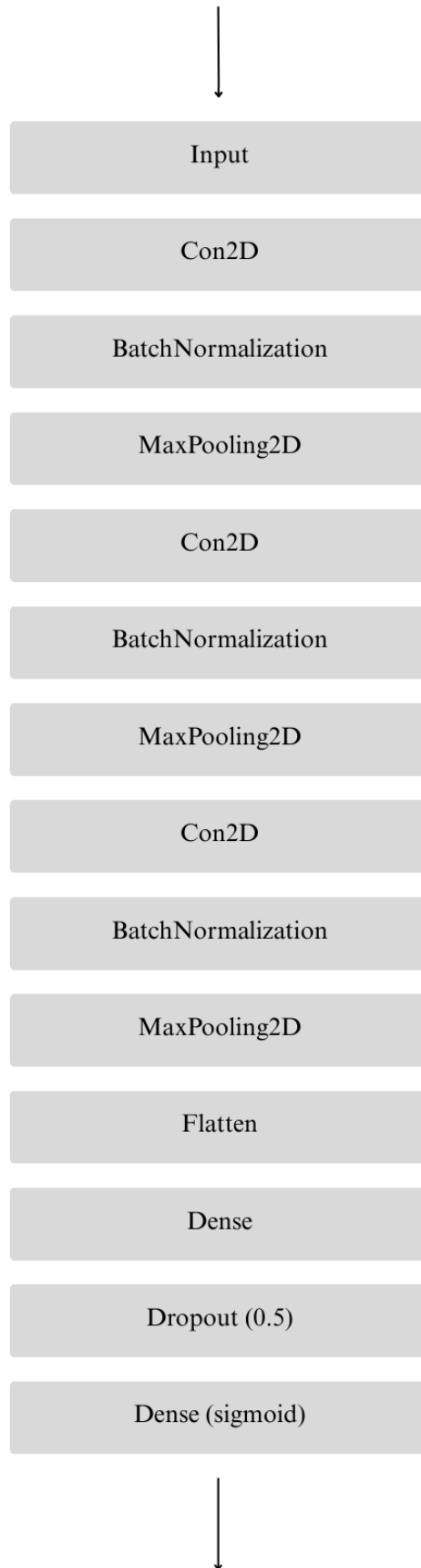


Figura 5.7: Capas CNN

También se han incluido capas de normalización por lotes *BatchNormalization*, que ayudan a que el entrenamiento sea más rápido y estable, ya que ajustan automáticamente las activaciones de las neuronas. Además, se ha añadido una capa *Dropout* con una tasa del 50 % para reducir el riesgo de sobreajuste ya que evita que la red memorice los datos de entrenamiento.

Después de todas estas transformaciones, se utiliza una capa Dense con activación *sigmoid*. Esta capa es la encargada de producir la salida final del modelo, que en este caso es de tipo multilabel, es decir, una misma muestra puede tener varias etiquetas (en nuestro caso géneros) activas a la vez. Por esta razón, también se ha elegido como función de pérdida *binary_crossentropy*, que se ajusta mejor a este tipo de problemas donde puede haber varias clases verdaderas por cada entrada. Todos los fundamentos teóricos sobre las neuronas utilizadas pueden consultarse en la sección 2.

```
1 def create_model(input_shape, num_classes):
2     model = Sequential()
3     model.add(Input(shape=input_shape))
4     model.add(Conv2D(32, (3, 3), activation='relu'))
5     model.add(BatchNormalization())
6     model.add(MaxPooling2D(2, 2))
7     model.add(Conv2D(64, (3, 3), activation='relu'))
8     model.add(BatchNormalization())
9     model.add(MaxPooling2D(2, 2))
10    model.add(Conv2D(128, (3, 3), activation='relu'))
11    model.add(BatchNormalization())
12    model.add(MaxPooling2D(2, 2))
13    model.add(Flatten())
14    model.add(Dense(256, activation='relu'))
15    model.add(Dropout(0.5))
16    model.add(Dense(num_classes, activation='sigmoid'))
17
18    model.compile(optimizer='adam',
19                  loss='binary_crossentropy',
20                  metrics=['accuracy', tf.keras.metrics.AUC(), tf.keras.
21                        metrics.Precision(), tf.keras.metrics.Recall()])
22    return model
```

Extracto de código 5.1: Modelo CNN

Como se puede ver en el fragmento de código 5.2, durante el entrenamiento del modelo, se estableció un máximo de 30 épocas; también se utilizó la técnica de *early stopping* la cual detiene automáticamente el proceso de entrenamiento cuando el rendimiento en el conjunto de validación deja de mejorar durante un número determinado de épocas consecutivas. Esta estrategia es fundamental para prevenir el sobreajuste, es decir, que el modelo aprenda demasiado bien los datos de entrenamiento a costa de disminuir su capacidad de generalización a nuevos datos.

```
1 image_size = (128, 128)
2 batch_size = 64
```

```

3 epochs = 30
4 num_folds = 5
5
6 kf = KFold(n_splits=num_folds, shuffle=True, random_state=42)
7
8 for train_index, val_index in kf.split(data):
9     x_train, x_val = data[train_index], data[val_index]
10    y_train, y_val = labels[train_index], labels[val_index]
11
12    model = create_model(input_shape=input_shape, num_classes=
13                          num_classes)
14    early_stopping = EarlyStopping(patience=5, restore_best_weights
15                                  =True)
16
17    model.fit(x_train, y_train,
18              validation_data=(x_val, y_val),
19              epochs=epochs,
20              batch_size=batch_size,
21              callbacks=[early_stopping],
22              verbose=0)
23
24    scores = model.evaluate(x_val, y_val, verbose=0)

```

Extracto de código 5.2: Entrenamiento con early stopping

6. Validación

6.1. Introducción

En este capítulo explicaremos el procedimiento seguido para validar el modelo entrenado, así como las métricas utilizadas y su evolución a lo largo del proceso de entrenamiento.

6.2. Validación del modelo

Con el objetivo de obtener resultados más robustos y evitar que dependan de una única división de los datos, se utilizó la técnica de validación cruzada con 5 folds (K-Fold Cross Validation). Esta técnica divide el conjunto de datos en cinco partes distintas y realiza cinco entrenamientos diferentes, utilizando en cada uno cuatro partes para entrenar y una para validar, como se muestra en la figura 6.1.

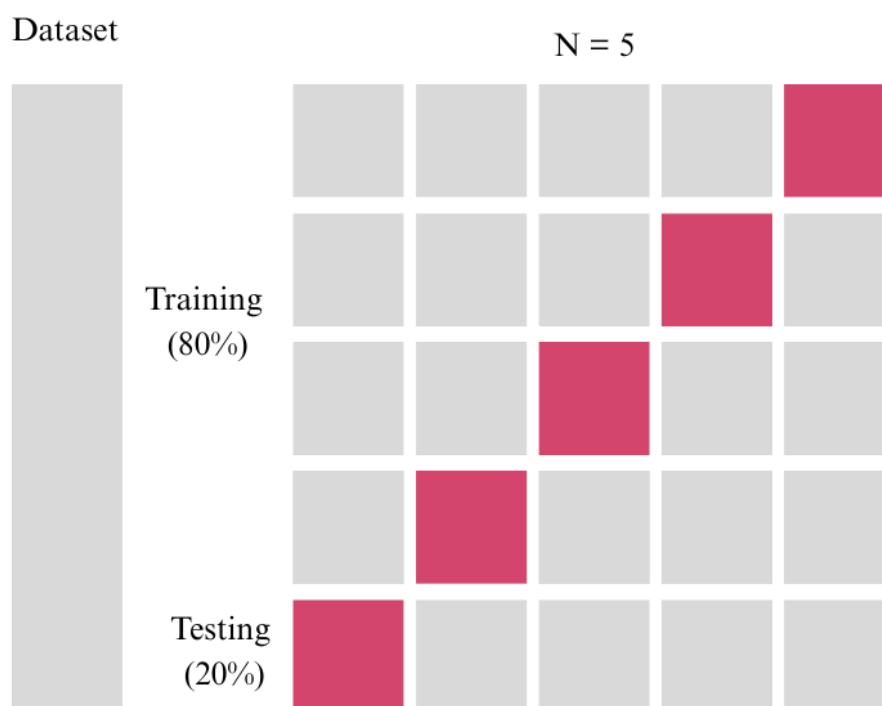


Figura 6.1: K-Fold Cross Validation

Para evaluar el rendimiento del modelo entrenado, se han utilizado varias métricas. Inicialmente, se utilizó únicamente la métrica de precisión general *accuracy* pero no es adecuada en problemas de clasificación multilabel, ya que solo se considera correcta una predicción si coincide exactamente con todas las etiquetas reales. Por esta

razón, se añadieron métricas adicionales más representativas del comportamiento del modelo en este tipo de tareas.

Además, se aplicó la técnica de *early stopping* con una paciencia de 5 épocas. Esto significa que el entrenamiento se detiene automáticamente si el modelo no muestra mejoras en la métrica de validación durante cinco épocas seguidas. Esta medida ayuda a evitar el sobreentrenamiento (overfitting), que ocurre cuando el modelo se ajusta demasiado a los datos de entrenamiento y pierde capacidad para generalizar sobre datos nuevos.

En nuestro caso, el entrenamiento se detuvo en la época 15, lo cual indica que el modelo alcanzó su punto óptimo de rendimiento antes de completar todas las épocas previstas. Este punto se determinó observando las métricas de validación como la pérdida (loss), la precisión (accuracy), la precisión positiva (precision), la recuperación (recall) y el AUC (Área Bajo la Curva ROC). A partir de la época 15, dichas métricas dejaron de mostrar mejoras significativas, lo cual activó la condición del *early stopping*.

En la tabla 6.1 y se muestra la evolución de las principales métricas durante las primeras 15 épocas del entrenamiento del modelo:

Epoch	Acc	AUC	Loss	Prec	Rec
1	0.4638	0.9117	0.1557	0.7121	0.3333
2	0.6219	0.9402	0.1281	0.9159	0.3727
3	0.6360	0.9470	0.1285	0.9040	0.3296
4	0.6675	0.9598	0.1074	0.8887	0.4896
5	0.6243	0.9502	0.1185	0.8367	0.4930
6	0.6806	0.9627	0.1021	0.8672	0.5420
7	0.6493	0.9540	0.1133	0.8567	0.5110
8	0.6904	0.9607	0.1051	0.8850	0.5212
9	0.6909	0.9662	0.0986	0.8861	0.5396
10	0.6924	0.9658	0.0971	0.8555	0.5870
11	0.7046	0.9614	0.1024	0.8617	0.5852
12	0.7034	0.9611	0.1035	0.8535	0.5819
13	0.6559	0.9297	0.1701	0.8269	0.5708
14	0.6965	0.9558	0.1102	0.8584	0.5473
15	0.6940	0.9618	0.1023	0.8641	0.5533

Cuadro 6.1: Evolución de métricas de validación (Épocas 1–15)

Los resultados obtenidos durante el entrenamiento reflejan una evolución progresiva del rendimiento del modelo a lo largo de las 15 épocas. En las primeras iteraciones, se observa un rápido incremento tanto en la precisión como en el AUC, lo que indica que el modelo comienza a aprender patrones útiles desde fases tempranas. Por ejemplo, ya en la época 2, la métrica AUC alcanza un valor de 0.9117 en entrenamiento.

Conforme avanza el entrenamiento, estas métricas continúan mejorando de forma sostenida. En la época 10, se alcanza una precisión del 85.55 % y una sensibilidad (recall) del 58.7 %, lo que muestra que el modelo no solo acierta con mayor frecuencia, sino que además es capaz de detectar un mayor número de verdaderos positivos. Esto

es especialmente relevante en tareas como la nuestra, ya que cada fragmento puede tener varios géneros simultáneamente.

Sin embargo, a partir de la época 12 comienzan a aparecer indicios de sobreajuste. Aunque las métricas de entrenamiento continúan mejorando (con un AUC superior al 92 %), los valores en el conjunto de validación muestran retrocesos, como se observa en la caída de la AUC de validación a 0.9297 en la época 13. Este patrón sugiere que el modelo comienza a memorizar los datos de entrenamiento, en lugar de generalizar correctamente a datos no vistos.

Gracias al mecanismo de *early stopping*, el entrenamiento se detuvo automáticamente en la época 15, evitando así que el modelo continuara con el sobreajuste. En ese punto, se alcanza un equilibrio con precisión (86.41 %) y unos valores robustos también en validación (precisión del 86.41 % y sensibilidad del 55.33 %). Estos resultados indican un buen equilibrio entre las distintas métricas y validan la eficacia del entrenamiento.

Por último, en la figura 6.2 se ha representado la evolución de las principales métricas de validación a lo largo de las distintas épocas de entrenamiento:

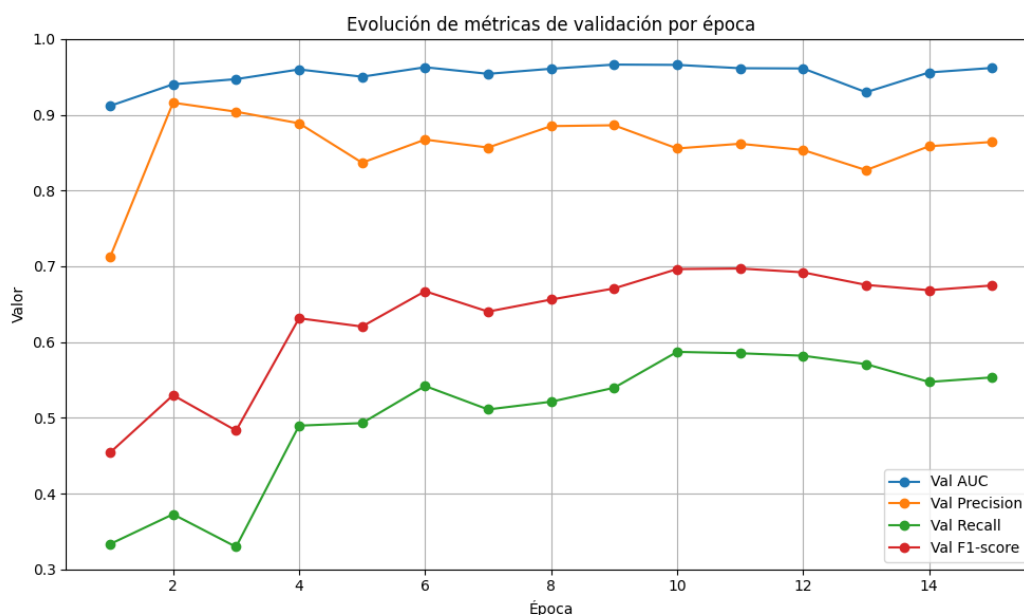


Figura 6.2: Evolución de las métricas de validación por época

Como vemos, el AUC se mantiene elevado durante todo el entrenamiento, lo cual indica una buena capacidad del modelo para distinguir entre las clases. La precisión presenta una ligera caída a mitad del entrenamiento, pero en general se mantiene estable en valores altos. Por otro lado, el recall y el F1-score muestran una mejora progresiva durante las primeras épocas, estabilizándose después alrededor de los valores óptimos obtenidos. Estos resultados respaldan el uso del *early stopping*, ya que se aprecia una ligera tendencia al sobreajuste a partir de la época 12, donde algunas métricas de validación comienzan a descender mientras que las de entrenamiento siguen mejorando.

6.3. Comparativa con resultados de estudios previos

En la Tabla 6.2 se compara el rendimiento de mi modelo frente a trabajos previos (explicados en la sección 3.7). El rendimiento se evalúa utilizando el área bajo la curva ROC (AUC) y, como se observa, mi modelo alcanza un AUC de 0.9618, superando de forma notable a los modelos anteriores, cuyos valores oscilan entre 0.861 y 0.898.

Referencia	Año	Modelo	AUC
-	2025	Mi modelo	0.9618
[7]	2016	FC-4	0.894
[35]	2015	Bag of features y RBM	0.888
[11]	2014	Convoluciones 1D	0.882
[45]	2014	Transferencia de aprendizaje	0.88
[10]	2012	Enfoque multi-escala	0.898
[18]	2011	Pooling MFCC	0.861

Cuadro 6.2: Comparativa del rendimiento entre el modelo propuesto y trabajos previos

Esta mejora puede deberse a varias diferencias respecto a los enfoques previos.

La primera diferencia podría ser que he implementado un preprocesamiento distinto al agrupar los géneros similares y, además, fragmento los audios en segmentos de 3 segundos y utilizo ventanas deslizantes, lo que incrementa el número de muestras disponibles para el entrenamiento y permite al modelo aprender patrones temporales más finos.

Otra diferencia podría ser que mi modelo utiliza tres bloques de convolución seguidos de normalización por lotes y max pooling, finalizando con capas densas y dropout para regularización. Esto contrasta con la arquitectura de Choi et al. [7], que empleaba capas convolucionales mucho más profundas y procesaba audios completos de 30 segundos sin ventanas deslizantes, lo que puede limitar la capacidad de generalización y aumentar el riesgo de sobreajuste.

7. Conclusiones y trabajo a futuro

En este trabajo se presenta una propuesta para la clasificación de fragmentos musicales en distintos géneros de espectrogramas generados a partir de archivos de audio y procesados mediante redes neuronales convolucionales.

El uso del dataset *MagnaTagATune* ha sido fundamental, no solo por su riqueza en etiquetas, sino también porque me ha obligado a enfrentarme a la realidad del desbalanceo de clases, un problema habitual en tareas de clasificación musical. Para mitigar este efecto, opté por agrupar géneros con características similares, lo que me permitió trabajar con menos clases más equilibradas y así, obtener resultados más robustos. Este enfoque ha demostrado ser efectivo, especialmente al comparar el rendimiento de mi modelo con el de estudios previos, donde la diferencia en las métricas evidencia la importancia de adaptar el preprocesamiento y la estructura de los datos a las necesidades concretas del problema.

En cuanto a los espectrogramas, he podido experimentar con distintas formas de generarlos y he comprobado cómo su uso como entrada para redes convolucionales potencia la capacidad del modelo para captar patrones relevantes. La representación tiempo-frecuencia que ofrecen resulta especialmente útil para el análisis musical, ya que permite que la red detecte matices que serían difíciles de identificar a partir de la señal de audio en crudo.

La naturaleza multilabel de mi clasificación me llevó a investigar y utilizar varias métricas de evaluación. La validación cruzada con cinco *folds*, junto con el uso de *early stopping*, ha sido clave para evitar el sobreajuste y asegurar que los resultados obtenidos sean realmente representativos y no fruto de una partición afortunada de los datos.

Trabajar con redes neuronales convolucionales ha sido una oportunidad para profundizar en su funcionamiento y experimentar con diferentes arquitecturas y configuraciones. No solo he aprendido a ajustar parámetros y a interpretar los resultados, sino que también he podido comparar mi enfoque con otros más complejos, comprobando que, en ocasiones, una arquitectura más sencilla y un buen preprocesamiento pueden superar a modelos mucho más profundos y pesados, especialmente cuando los datos son limitados o ruidosos.

En relación con la aplicabilidad de este trabajo, creo que lo desarrollado aquí podría tener un impacto real en entornos empresariales, especialmente en plataformas de streaming musical, sistemas de recomendación o herramientas de catalogación automática. La flexibilidad del enfoque permite adaptarlo a otros tipos de datos relacionados con el audio, como la clasificación de emociones, la detección de instrumentos o incluso la segmentación de eventos sonoros en otros contextos, como el análisis de sonido ambiental o la monitorización de calidad en entornos industriales.

Aunque el uso de *MagnaTagATune* ha sido muy valioso, sería interesante ampliar el estudio a otros datasets, como GTZAN, para comprobar la capacidad de generalización del modelo y explorar posibles mejoras. Además, siempre queda margen para probar nuevas arquitecturas, incorporar técnicas de interpretabilidad que ayuden a en-

tender mejor las decisiones del modelo o incluso adaptar el sistema para funcionar en tiempo real, abriendo la puerta a aplicaciones prácticas en streaming o en dispositivos con recursos limitados.

En definitiva, este trabajo me ha permitido comprobar que la combinación de un buen preprocesamiento, una arquitectura adaptada y una evaluación rigurosa puede marcar la diferencia en tareas complejas como la clasificación musical multilabel.

A. Bibliografía

- [1] Safaa Allamy and Alessandro Lameiras Koerich. 1d cnn architectures for music genre classification. *arXiv preprint arXiv:2105.07302*, 2021. URL <https://arxiv.org/abs/2105.07302>.
- [2] Aphex. Inteligencia artificial en la enfermedad de parkinson y otros trastornos del movimiento - scientific figure on researchgate, 2025. URL https://www.researchgate.net/figure/Figura-1-Esquema-de-la-arquitectura-de-las-redes-neuronales-convolucionales-Modifi-fig1_371564848.
- [3] Hareesh Bahuleyan. Music genre classification using machine learning techniques. *arXiv preprint arXiv:1804.01149*, 2018. URL <https://arxiv.org/abs/1804.01149>.
- [4] N. Bala Ganesh et al. Genrenet: A deep based approach for music genre classification. *SN Computer Science*, 5:1135, 2024. URL <https://doi.org/10.1007/s42979-024-03493-x>.
- [5] O. Barcelona Fernández. Deep learning. redes neuronales convolucionales aplicadas a la detección de la malaria. *Universidad de Zaragoza*, page 41, 2024. URL <https://zagan.unizar.es/record/149481/files/TAZ-TFG-2024-3499.pdf>.
- [6] Julian Blackmore. ¿qué es el espectrograma?, 2023. URL <https://emastered.com/es/blog/what-is-spectrogram>.
- [7] Keunwoo Choi, György Fazekas, Mark Sandler, and Kyunghyun Cho. Convolutional recurrent neural networks for music classification. *arXiv preprint arXiv:1606.00298*, 2016.
- [8] Keunwoo Choi, György Fazekas, Mark Sandler, and Kyunghyun Cho. A tutorial on deep learning for music information retrieval. *arXiv preprint arXiv:1709.04396*, 2017.
- [9] Gobierno de España. Qué es la inteligencia artificial, 2023. URL <https://planderecuperacion.gob.es/noticias/que-es-inteligencia-artificial-ia-prtr>.
- [10] Sander Dieleman and Benjamin Schrauwen. Multiscale approaches to music audio feature learning. In *Proceedings of the 14th International Society for Music Information Retrieval Conference, ISMIR 2013*, pages 116–121, Curitiba, Brazil, 2013.
- [11] Sander Dieleman and Benjamin Schrauwen. End-to-end learning for music audio. In *2014 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 6964–6968. IEEE, 2014.

- [12] Mingwen Dong. Convolutional neural network achieves human-level accuracy in music genre classification. *arXiv preprint arXiv:1802.09697*, 2018. URL <https://arxiv.org/abs/1802.09697>.
- [13] Timothy Dozat. Incorporating nesterov momentum into adam. *arXiv preprint*, 2016. URL <https://arxiv.org/abs/1609.04747>. arXiv:1609.04747.
- [14] P. Dwivedi and B. Islam. Generative adversarial networks based framework for music genre classification. *SN Computer Science*, 5:1149, 2024. URL <https://doi.org/10.1007/s42979-024-03531-8>.
- [15] Glassdoor. Sueldos de ingeniero de software, 2025. URL https://www.glassdoor.es/Sueldos/software-engineer-suelo-SRCH_K00,17.htm.
- [16] Hanlin Gu, Yin Xian, Ilona Christy Unarta, and Yuan Yao. Generative adversarial networks for robust cryo-em image denoising. *arXiv preprint arXiv:2008.07307*, 2020. URL <https://arxiv.org/abs/2008.07307>.
- [17] B Gómez Pujante. Redes convolucionales. aplicación a la clasificación de imágenes médicas. *Universidad Miguel Hernández de Elche*, page 94, 2023. URL <https://dspace.umh.es/jspui/bitstream/11000/30233/1/TFG-G%C3%B3mez%20Pujante%2C%20Bego%C3%B1a.pdf>.
- [18] Philippe Hamel, Simon Lemieux, Yoshua Bengio, and Douglas Eck. Temporal pooling and multiscale learning for automatic annotation and ranking of music audio. In *Proceedings of the 12th International Society for Music Information Retrieval Conference, ISMIR 2011*, pages 729–734, Miami, Florida, USA, 2011.
- [19] Red Hat. ¿qué es el aprendizaje automático?, 2023. URL <https://www.redhat.com/es/topics/ai/what-is-machine-learning>.
- [20] K. He, X. Zhang, S. Ren, and J. Sun. Deep residual learning for image recognition. In *CVPR*, 2016.
- [21] Shawn Hershey, Sourish Chaudhuri, Daniel P. W. Ellis, Jort F. Gemmeke, Aren Jansen, Ron Moore, Manoj Plakal, David Platt, Rif A. Saurous, Brian Seybold, et al. Cnn architectures for large-scale audio classification. In *2017 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 131–135. IEEE, 2017.
- [22] G. Hinton. Neural networks for machine learning: Lecture 6a rmsprop & lecture 6b momentum, 2012.
- [23] G. Huang, Z. Liu, L. Van Der Maaten, and K. Q. Weinberger. Densely connected convolutional networks. In *CVPR*, 2017. URL <https://www.sciencedirect.com/journal/neurocomputing>.
- [24] IBM. Aprendizaje no supervisado, 2023. URL <https://www.ibm.com/es-es/topics/unsupervised-learning>.

- [25] P. Jain, A. Sar, T. Choudhury, V. Singh, and K. Kotecha. Differentiation of music genre from an audio file using neural networks. *AI Technologies for Information Systems and Management Science*, 1136:482–490, 2024. doi: 10.1007/978-3-031-70789-6_40. URL https://link.springer.com/chapter/10.1007/978-3-031-70789-6_40.
- [26] Ee Hern Kheng, Chia Pao Liew, Tianhao Lan, and Kim Geok Tan. Advancing handwritten musical notation recognition using deep learning: A convolutional neural network-based approach with improved accuracy. *International Journal of Pattern Recognition and Artificial Intelligence*, 38(3):2452007, 2024. doi: 10.1142/S0218001424520074.
- [27] Taejun Kim, Jongpil Lee, and Juhan Nam. Sample-level cnn architectures for music auto-tagging using raw waveforms. In *ICASSP 2018 - IEEE International Conference on Acoustics, Speech and Signal Processing*, pages 366–370. IEEE, 2018. URL <https://arxiv.org/abs/1710.10451>.
- [28] D. P. Kingma and J. Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.
- [29] Edith Law and Luis Von Ahn. Magnatagatune dataset. <https://mirg.city.ac.uk/datasets/magnatagatune/>, 2009.
- [30] Hao Li, Gopi Krishnan Rajbahadur, Dayi Lin, Cor-Paul Bezemer, and Zhen Ming Jiang. Keeping deep learning models in check: A history-based approach to mitigate overfitting. *arXiv preprint arXiv:2401.10359*, 2024. URL <https://arxiv.org/abs/2401.10359>.
- [31] Y. Li, Y. Zhang, X. Wang, and Q. Liu. Accuracy assessment in convolutional neural network-based deep learning remote sensing studies—part 1: Literature review. *Remote Sensing*, 13(13):2450, 2021. doi: 10.3390/rs13132450.
- [32] Caifeng Liu, Lin Feng, Guochao Liu, Huibing Wang, and Shenglan Liu. Bottom-up broadcast neural network for music genre classification. *arXiv preprint arXiv:1901.08928*, 2019. URL <https://arxiv.org/abs/1901.08928>.
- [33] Zhuang Liu, Zhiqiu Xu, Joseph Jin, Zhiqiang Shen, and Trevor Darrell. Dropout reduces underfitting. *arXiv preprint arXiv:2303.01500*, 2023. URL <https://arxiv.org/abs/2303.01500>.
- [34] Christopher Mahlich, Tobias Vente, and Joeran Beel. From theory to practice: Implementing and evaluating e-fold cross-validation. *arXiv preprint arXiv:2410.09463*, 2024. URL <https://arxiv.org/abs/2410.09463>.
- [35] Juhan Nam, Jorge Herrera, and Kyogu Lee. A deep bag-of-features model for music auto-tagging. *arXiv preprint arXiv:1508.04999*, 2015.
- [36] Ndiatenda Ndou, Ritesh Ajoodha, and Ashwini Jadhav. Music genre classification: A review of deep-learning and traditional machine-learning approaches. In *2021 IEEE International IOT, Electronics and Mechatronics Conference (IEMTRONICS)*, pages 1–6, 2021. doi: 10.1109/IEMTRONICS52119.2021.9422487.

- [37] Yurii Nesterov. A method for unconstrained convex minimization problem with the rate of convergence $o(1/k^2)$. *Doklady AN USSR*, 269(3):543–547, 1983.
- [38] Martín Ezequiel Paz, Guillermo Friedrich, and Christian Luis Galasso. Procesamiento de señal visualizado sobre un espectrograma. *Elektron: Ciencia y Tecnología en la Electrónica de Hoy*, 4(1):35–39, 2020. ISSN 2525-0159. URL <https://dialnet.unirioja.es/servlet/articulo?codigo=7468991>.
- [39] Lawrence R. Rabiner and Biing-Hwang Juang. *Fundamentals of speech recognition*. Prentice Hall.
- [40] Pol Martín Sahuja. Entendiendo la curva roc y el auc: dos medidas del rendimiento de un clasificador binario que van de la mano. <https://polmartisanahuja.com/entendiendo-la-curva-roc-y-el-auc-dos-medidas-del-rendimiento-de-un-clasificador-binario/>
- [41] SoldAI. Tipos de aprendizaje automático, 2021. URL <https://medium.com/soldai/tipos-de-aprendizaje-automatico-6413e3c615e2>.
- [42] S. S. Stevens, J. Volkman, and E. B. Newman. A scale for the measurement of the psychological magnitude pitch. *The Journal of the Acoustical Society of America*, 8(3):185–190.
- [43] Talent.com. Salario de ingeniero de software en españa, 2025. URL <https://es.talent.com/salary?job=ingeniero+de+software>.
- [44] Telefónica. Aprendizaje supervisado: definición y aplicaciones, 2023. URL <https://www.telefonica.com/es/sala-comunicacion/blog/aprendizaje-supervisado-definicion-aplicaciones/>.
- [45] Aaron Van Den Oord, Sander Dieleman, and Benjamin Schrauwen. Transfer learning by supervised pretraining for audio-based music classification. In *Proceedings of the 15th International Society for Music Information Retrieval Conference, ISMIR 2014*, pages 29–34, Taipei, Taiwan, 2014.
- [46] Minz Won, Andres Ferraro, Dmitry Bogdanov, and Xavier Serra. Evaluation of cnn-based automatic music tagging models. Zenodo, 2020. URL <https://zenodo.org/record/3898838>.
- [47] Eberhard Zwicker and Hugo Fastl. *Psychoacoustics: Facts and models*. Springer Science & Business Media, 2013.